

VIETNAMESE – GERMAN UNIVERSITY

**Faculty of Engineering
Computer Science Program**

**Design and Pilot Protocol of a Next.js Boilerplate
Framework for AI-Assisted SaaS Development**

Full name: Do Hoang Trieu

Student ID: 10422120

First supervisor: Dr. Le Lam Son

Second supervisor: Dr. Tran Huu Tam

BACHELOR THESIS

Submitted in partial fulfillment of the requirements for the degree of Bachelor Engineering
in study program Computer Science, Vietnamese - German University, 2026

Ho Chi Minh City, Viet Nam

Declaration

I, Do Hoang Trieu, hereby declare that this thesis, entitled “Design and Pilot Protocol of a Next.js Boilerplate Framework for AI-Assisted SaaS Development,” is my own original work. All sources of information have been properly acknowledged and referenced.

In preparing this thesis, I used AI tools to assist with code generation, prose drafting, and information research. I reviewed the output against the original sources and the SmartKit codebase, and take full responsibility for the content.

This thesis has not been submitted, in whole or in part, for the award of any other academic degree at this or any other institution. The author agrees that this thesis, upon acceptance, will be archived in the library of Vietnamese–German University and made available for reference use in accordance with the university’s regulations.

Acknowledgements

This thesis is the product of several people who gave me their time, their patience, and more faith in my work than I could always return.

My family deserves the first and the largest thank-you. They absorbed every long weekend of writing, every anxious phone call, and the cost of one more year of schooling without making me feel the weight of any of it. I hope the result makes that easier to bear.

I am grateful to my first supervisor, **Dr. Le Lam Son**, for his steady guidance through every stage of this project. He was the person I went to when the work felt stuck, and he was generous with his time in a way I will not forget. **Dr. Tran Huu Tam**, my second supervisor, gave me the same kind of attention whenever I needed it, and his perspective made several of the harder decisions in this thesis easier to make. I am thankful to both.

I also thank the **Faculty of Engineering** and the **Department of Computer Science** at the **Vietnamese–German University** for four years of teaching and for the academic environment in which this work was carried out.

Finally, my thanks go to my **university peers and friends**, the people who sat next to me through long evenings and short deadlines, and who reminded me that there is a life beyond a single document.

Abstract

AI coding assistants such as GitHub Copilot and Cursor AI are now in daily use, with substantial reported gains for boilerplate-heavy tasks: Peng et al. report a 55.8% completion-time reduction on a controlled HTTP-server task (Peng et al., 2023), and Yetistiren et al. find boilerplate generation the strongest use case among practitioners (Yetistiren et al., 2022). The benchmarks that dominate the literature, including HumanEval, MBPP+, APPS, Big-CodeBench, and LiveCodeBench for algorithms and SWE-bench for bug fixing, measure problem-solving and patch generation, not whether AI-generated code respects framework conventions. Server/Client boundary violations in Next.js App Router, for instance, fail at runtime in a way no static type check catches.

This thesis proposes a three-dimensional evaluation framework for AI-generated Next.js boilerplate and instantiates it in SmartKit, an open-source Next.js 16.2.12 boilerplate that acts as both the framework’s target codebase and a usable SaaS scaffold. The framework scores each generated feature on Structural Integrity, Type & Contract Safety, and Coverage & Completeness, each reported on a 0–5 scale. A pre-registered adaptive pilot ran 8 times on one feature: $n = 5$ in Condition A (SmartKit) after a mid-study extension, $n = 3$ in Condition B (convention-naive scaffold). One of three directional predictions is confirmed outright (structural placement); one is confirmed as an instruction-following asymmetry rather than as a convention-encoding benefit (Zod-derived FlagKey type); and one is asymmetric, confirmed in Condition A and refuted in Condition B, and is reported as such rather than collapsed into a single verdict.

The pilot is feasibility-stage evidence rather than an inferential test. Per-arm samples are too small for inferential statistics, and three protocol adaptations are disclosed: an adaptive extension applied to Condition A only, a tool substitution for runs 6–8, and two-clause reporting of the asymmetric prediction introduced after data collection. Larger-N replication, cross-tool validation, and developer-participant studies remain future work.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
1 Introduction and Motivation	1
1.1 Overview	1
1.2 The Problem: AI Convention Violations in Next.js	2
1.3 Research Objectives and Questions	2
1.4 Scope and Limitations	6
2 Background	8
2.1 AI Coding Assistants	8
2.2 Next.js Framework Architecture	8
2.3 SaaS Boilerplate Patterns	9
3 Related Work	10
3.1 Code Generation Benchmarks	10
3.2 Empirical Studies on Developer Productivity	11
3.3 Evaluation Methodology	12
3.4 Research Gap Summary	12
3.5 Literature Search Methodology	13
3.6 Alternative Approaches to AI-Codebase Alignment	13
4 Proposed Solution	16
4.1 Design Rationale	16
4.2 Feature-Based Modular Architecture	17
4.3 Server/Client Boundary Conventions	17
4.4 Database Schema Patterns	18
4.5 Type-Safe API Contracts	19
5 Implementation	20
5.1 Technology Stack	20
5.1.1 Stack Selection Rationale	20
5.1.2 Dependency Version Pins	20
5.2 Project Structure	21
5.3 Key Implementation Patterns	23
5.4 Billing and SePay Integration	23
6 Evaluation	24
6.1 Evaluation Framework	24
6.1.1 Three Dimensions	24
6.1.2 Design Rationale and Trade-offs	26
6.1.3 Evaluation Metrics	27

6.2	Pilot Methodology	28
6.2.1	Conditions	28
6.2.2	Standardized Prompt	30
6.2.3	Run Schedule and Verdict Rule	31
6.2.4	Scoring Rubric	31
6.2.5	Exclusion Criteria	34
6.3	Empirical Results	34
6.3.1	Per-run Scores	34
6.3.2	Per-condition Aggregate	35
6.3.3	Automated vs. Manual Cross-Check	35
6.3.4	Directional Prediction Verdicts	37
6.4	Discussion of Trade-offs	38
6.4.1	Where the framework wins: DP1 as a measurement success	38
6.4.2	Where the framework loses	39
6.4.3	Statistical context: $n = 5$ and $n = 3$ are not inferential	40
6.5	Pre-registration and Amendments	40
6.5.1	Lock, Amendment Procedure, and Snapshot Limitation	40
6.5.2	Pilot Run Log and Amendment Record	41
6.6	Threats to Validity	41
6.6.1	External Validity (Generalizability)	41
6.6.2	Construct Validity	42
6.6.3	Internal Validity	42
6.6.4	Temporal Validity	44
6.6.5	Conclusion Validity	44
6.6.6	Instrumentation	44
7	Conclusion and Future Work	46
7.1	Summary	46
7.2	Limitations and Threats to Validity	46
7.3	Diffusion Pathway and Adoption Risks	49
7.4	Future Work	50
	Bibliography	54

List of Figures

1.1 AI-generated Next.js App Router code violating Server/Client boundaries by passing event handlers across component types, with the CORRECT panel showing the composition-pattern fix. 5

1.2 Evaluation-framework architecture (target design) for the design-stage 150-artifact corpus (5 tools × 3 scenarios × 10 prompts); this thesis executes an 8-run subset (1 tool × 1 feature × 8 runs). 6

List of Tables

3.1	Qualitative positioning of SmartKit against established Next.js starter templates. Cells describe documented features, not measured performance; cell-level claims were transcribed from each template’s own documentation.	14
5.1	Canonical dependency pins for SmartKit at the pilot-evaluation snapshot. Versions match <code>smartkit/package.json</code> ; <code>better-auth</code> lists the latest 1.6.x line as of the snapshot.	21
6.1	Reference coverage checklist for the <code>isEnabled</code> feature flag utility.	26
6.2	Condition A (SmartKit at full setup) vs. Condition B (convention-naive scaffold) used in the pilot evaluation.	29
6.3	Structural Integrity scoring rubric (0–5).	32
6.4	Type Safety sub-score rubric (0–5).	32
6.5	Contract Safety sub-score rubric (0–5).	33
6.6	Coverage & Completeness scoring rubric (0–5).	33
6.7	Exclusion criteria for pilot runs.	34
6.8	Per-run pilot scores (Str / T&C / Cov, 0–5 rubric) across 8 runs: Condition A ($N = 5$, Runs 4–8) vs. Condition B ($N = 3$, Runs 1–3). Tool column shows the Cursor AI mode; per-minute timestamps and time-to-correct live in the pilot audit trail released in the thesis GitHub repository (<code>REPLICATION.md</code>). The Verdict column is the directional-prediction outcome under the binary rule (§6.2.3).	36
6.9	Per-run automated vs. manual score cross-check across all eight pilot runs; maximum per-dimension absolute delta = 2 across 24 (run, dimension) pairs, driven entirely by the post-correction Coverage cells (Runs 4–8; see the “Note on cross-check methodology” below).	37

1. Introduction and Motivation

The chapter opens with the recent wave of AI coding assistants and the parallel shift in Next.js toward the App Router, then identifies the absence of an evaluation framework for framework-specific convention adherence. From that gap the chapter derives the research objectives, questions, and hypotheses that structure the rest of the thesis, and closes with the scope within which the SmartKit framework is designed and evaluated.

1.1 Overview

Controlled studies report substantial productivity gains from AI coding assistants. Peng et al. showed that developers using GitHub Copilot completed tasks 55.8% faster than control groups (Peng et al., 2023), while Yetistiren et al. found AI tools particularly effective for boilerplate-generation tasks (Yetistiren et al., 2022). Next.js has emerged as the dominant React meta-framework for production web applications, with its App Router architecture introduced in version 13 fundamentally changing application structure through React Server Components and explicit Server/Client boundaries (Vercel, 2024). The failure mode that motivates this thesis is concrete (Figure 1.1): when an AI coding assistant generates a Server Component that imports a Client Component and passes an event handler as a prop, Next.js produces the runtime error “Event handlers cannot be passed to Client Component props”—a failure that `tsc --noEmit` cannot catch because it has no type-level representation. During SmartKit’s development, recurring AI generation errors were observed firsthand in Next.js App Router code, including authentication security flaws, component boundary violations, and outdated API usage from older training data; these are development-time observations rather than measured prevalence figures. Without systematic evaluation, developers cannot assess whether AI-generated Next.js code adheres to framework conventions or will fail in production.

Existing benchmarks address algorithmic problem-solving and bug-fixing capabilities but not framework convention adherence. HumanEval evaluates Python code generation for standalone functions (Chen et al., 2021); SWE-bench assesses AI ability to resolve GitHub issues (Jimenez et al., 2024); a systematic review of 395 LLM-for-SE studies found that framework-convention adherence is not separately tracked as an evaluation category, with benchmark evaluation (60%) dominating over human evaluation (40%) (Hou et al., 2024). None of these capture the specific architectural requirements of Next.js App Router, such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions. The gap means developers have no quantitative basis for comparing AI tools on framework-specific tasks.¹

Intended users. SmartKit targets solo developers and small teams (2–5 members) working on Next.js projects with AI coding assistants. They are intermediate developers who already understand React and TypeScript fundamentals and want to reduce boilerplate setup time while enforcing consistent conventions in AI-assisted development. The framework supports incremental adoption: teams can apply the feature-based folder structure and

¹RigorBench (Madiraju & Madiraju, 2026), an as-yet-unreviewed preprint with a self-disclosed author conflict of interest (the authors co-developed the Agent-Rigor framework that RigorBench evaluates), addresses an adjacent concern—engineering process discipline—and is referenced here for completeness rather than as a load-bearing citation for the gap claim above.

Server/Client boundary conventions without committing to the full stack.

1.2 The Problem: AI Convention Violations in Next.js

This thesis addresses a concrete problem: AI coding assistants consistently generate Next.js App Router code that violates framework conventions, producing runtime errors and technical debt, yet no evaluation framework exists to measure this adherence. Three principal challenges shape the framework design.

First, framework conventions are implicit and distributed across documentation, error messages, and community patterns. Enumerating them into testable metrics is itself an open problem. Second, AI-generated code failures in Next.js manifest at runtime rather than compile time — the Server/Client boundary error “Event handlers cannot be passed to Client Component props” only fires when the component is executed, so the framework needs dynamic analysis rather than static linting alone. Third, the three evaluation dimensions are designed to be orthogonal: a structural-integrity violation does not imply a type error, and a coverage gap operates independently of both. Collapsing them into a single composite score would hide which dimension is actually regressing. The orthogonality of the three dimensions is asserted at design stage and awaits empirical validation; the executed pilot’s per-arm sample is far below the $n \approx 30$ required to confirm that the dimensions vary independently in practice (Section 7.4 documents a Pearson correlation analysis as future work).

The focus on Next.js App Router is deliberate. First, market dominance makes it the primary target: community surveys consistently place Next.js among the most widely adopted React meta-frameworks (Vercel, 2024); Stack Overflow’s annual Developer Survey has tracked Next.js as the most-used React meta-framework since 2021 (Stack Overflow, 2025, Stack Overflow Developer Survey). Second, training-data mismatch creates measurable failures: AI tools generate outdated patterns from older training data that conflict with current Next.js conventions, particularly around Server/Client boundaries and async API changes. Third, practical feasibility applies: the author has existing Next.js expertise, enabling direct testing of enforcement mechanisms. The pilot targets Cursor AI as the primary case study because its AGENTS.md feature implements documentation-based guidance, which prior research on grounded prompting suggests can improve AI output quality (Barke et al., 2022; Zhang et al., 2023).

1.3 Research Objectives and Questions

While Next.js dominates React meta-framework adoption and AI coding assistants such as GitHub Copilot and Cursor AI have demonstrated substantial productivity gains, there is a distinct gap at the intersection of these two areas: a systematic, operationalizable evaluation framework that measures how closely AI-generated Next.js boilerplate code adheres to framework-specific conventions. Existing benchmarks—HumanEval, SWE-bench—address algorithmic problem-solving and bug-fixing capability, but none capture the architectural requirements of Next.js App Router such as Server/Client component boundaries, type-safe contracts, or file-system routing conventions (Chen et al., 2021; Jimenez et al., 2024). This thesis targets precisely that gap.

The primary motivation of this thesis is to bridge this gap by designing a three-dimensional evaluation framework that turns an implicit set of framework conventions into measurable, comparable scores, and by instantiating that framework in SmartKit, a structured

Next.js boilerplate that provides the necessary enforcement mechanisms and the evaluation target codebase.

The research pursues three objectives:

- **Design a three-dimensional evaluation framework.** Construct a framework that scores AI-generated Next.js boilerplate code along three dimensions: Structural Integrity, Type & Contract Safety, and Coverage & Completeness, so that convention adherence becomes a quantitative, comparable property of generated code.
- **Implement SmartKit as a structured architecture.** The thesis releases SmartKit, an open-source Next.js boilerplate that demonstrates feature-based modularity, explicit Server/Client boundaries, and type-safe contracts, providing both the enforcement mechanisms and the target codebase for the framework.
- **Conduct a pilot empirical case study.** The thesis applies the framework to a representative feature generated by Cursor AI under the SmartKit condition to demonstrate that the framework can produce illustrative, actionable scores on real generated code.

Consequently, this research seeks to answer the following research questions:

1. How can an evaluation framework be consistently designed to measure AI coding assistant adherence to Next.js App Router boilerplate conventions along the Structural Integrity, Type & Contract Safety, and Coverage & Completeness dimensions?
2. Can the resulting framework be operationalized into measurable metrics, and does it produce illustrative, actionable scores when applied to representative AI-generated Next.js features?

These objectives and questions give rise to two classes of claims, both documented as design-level pre-registrations to be exercised by the pilot (Section 6.2). The first class states **framework properties**—descriptive claims about whether the framework can measure the dimensions it targets. The second class states **directional predictions**—falsifiable claims about the sign of the score difference between conditions, to be reported as confirmed, refuted, or inconclusive per Section 6.2.

Framework properties. The pilot is designed to demonstrate that the framework can be operationalised on a real artifact:

- **FP1 (Structural Integrity).** The framework can measure file placement, Server/Client boundary usage, and naming patterns in AI-generated Next.js code, producing a 0–5 score per run on a representative feature.
- **FP2 (Type & Contract Safety).** The framework can measure TypeScript error counts and Zod-validation coverage in AI-generated Next.js code, producing a 0–5 score per run that combines a Type-Safety sub-score and a Contract-Safety sub-score via the weakest-link rule.
- **FP3 (Coverage & Completeness).** The framework can enumerate AI-generated files against a feature-specific checklist and produce a 0–5 per-run score that reflects required-artifact presence.

Directional predictions. Each directional prediction states the expected sign of the difference between Condition A (SmartKit at full setup) and Condition B (convention-naive scaffold) on a representative feature—the `isEnabled(flagKey, userId?)` utility. Predictions are evaluated as *confirmed* (all runs in the predicted direction), *refuted* (no runs in the predicted direction), or *inconclusive* (intermediate), per the adaptive-N protocol documented in Section 6.2.

- **DP1 (Structural).** Condition A is predicted to place the utility in the SmartKit feature-based layout in the majority of runs; Condition B is predicted to use a root-level or flat structure.
- **DP2 (Type & Contract).** Condition A is predicted to produce Zod-typed schema definitions in the majority of runs; Condition B is predicted to use raw string types.
- **DP3 (Coverage).** Condition A is predicted to include a centralized test file in the majority of runs; Condition B is predicted to omit a test file in the majority of runs.

Formal definitions and exact expected file paths appear in Section 6.2.

Framework properties and directional predictions serve different purposes. A framework property is a descriptive claim about the framework itself; it cannot be refuted by pilot data, only supported or unsupported. A directional prediction is a falsifiable claim about the artifact and is the only class of claim to which the binary *confirmed/refuted/inconclusive* verdict applies. The pilot protocol (Section 6.2) documents how each is reported. Full statistical validation—including larger sample sizes, cross-tool replication, and developer-participant studies—remains future empirical work.

Designing an evaluation framework for AI-generated Next.js boilerplate encounters three key challenges that distinguish it from existing code generation benchmarks. First, framework conventions are implicit and distributed across documentation, error messages, and community patterns, requiring deliberate extraction into enumerable, testable rules rather than relying on predefined quality criteria. Second, convention violations in Next.js App Router manifest primarily at runtime. Server/Client boundary violations produce errors only when executed, so evaluation must go beyond static analysis to capture dynamic execution behaviour. Third, the evaluation dimensions are orthogonal: structural integrity violations do not imply type errors, and coverage gaps operate independently of both, demanding a multi-dimensional framework rather than a single composite score. Pearson correlation analysis across runs (Section 7.4) would empirically test whether the three dimensions are genuinely orthogonal or merely weakly coupled; the executed pilot’s small per-arm sample precludes such analysis.

The contributions of this thesis are as follows. First, it proposes a three-dimensional evaluation framework (Structural Integrity, Type & Contract Safety, and Coverage & Completeness) for measuring how closely AI coding assistants adhere to Next.js App Router conventions. The framework addresses a gap left by existing benchmarks, which score algorithmic correctness rather than framework-specific quality. Second, it releases SmartKit, an open-source Next.js boilerplate demonstrating feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. SmartKit supplies both the conventions the framework measures and the target codebase it measures them on; all four Better Auth tables (user, session, account, verification), the billing feature (SePay integration with HMAC signature verification and idempotent webhook handling), and the email integration (Resend + React Email) are implemented at the current snapshot rather than deferred to future phases, supporting SmartKit’s

role as a functional SaaS boilerplate. Third, it releases the pilot protocol and the baseline comparison design as artefacts: a pre-registration document, a Condition A vs. Condition B comparison scaffold, a scoring rulebook, and a baseline-bootstrap script. The artefacts are concrete enough that another researcher can replicate the pilot without re-deriving the protocol from the thesis prose.

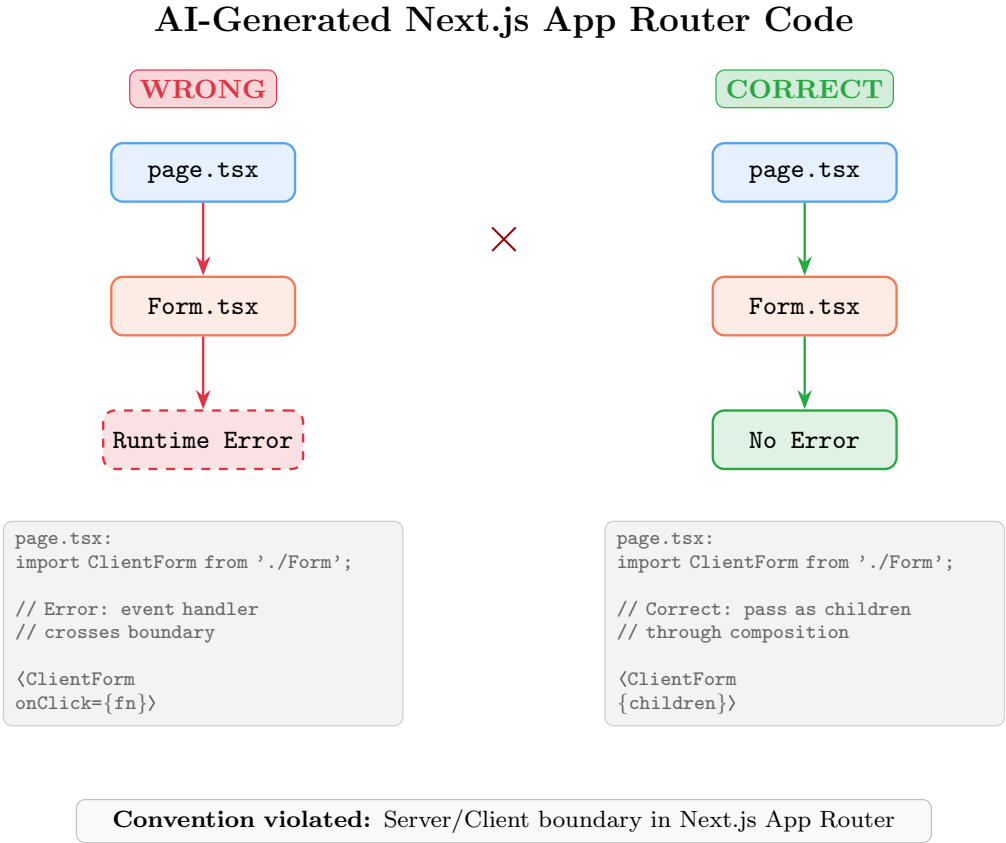


Figure 1.1: AI-generated Next.js App Router code violating Server/Client boundaries by passing event handlers across component types, with the CORRECT panel showing the composition-pattern fix.

Note. Boundary violations produce runtime errors that static analysis cannot detect; the CORRECT panel passes children as nested content inside `<ClientForm>...</ClientForm>`, not as a JSX prop attribute.

Figure 1.2 summarizes the resulting evaluation-framework architecture. Each of the three dimensions is paired with an automated measurement: Structural Integrity is measured through parsed-AST rule violations, Type & Contract Safety through TypeScript compiler output, and Coverage through a rule-coverage matrix checked against the convention catalog. The three measurements are applied to a shared corpus of AI-generated artifacts and produce a per-tool violation matrix, type-error rates, and coverage scores. Because the dimensions are deliberately independent, no single composite score can substitute for examining all three independently.

Evaluation Framework Architecture

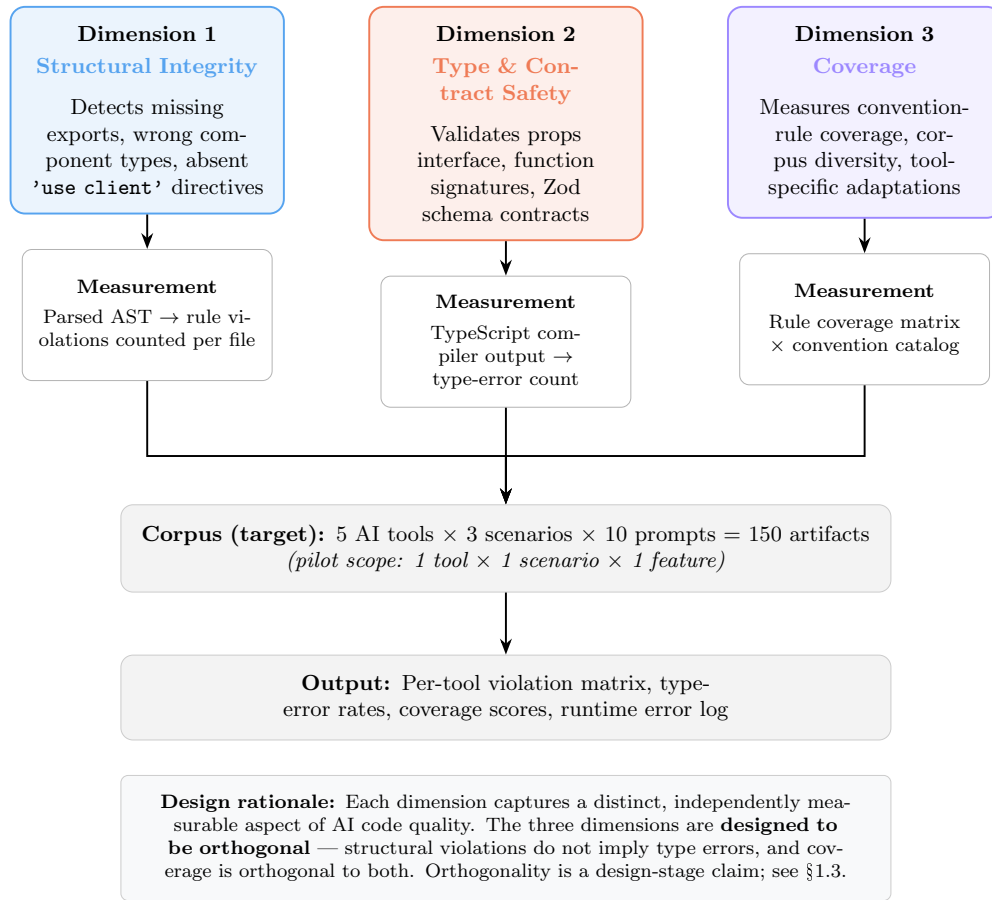


Figure 1.2: Evaluation-framework architecture (target design) for the design-stage 150-artifact corpus (5 tools × 3 scenarios × 10 prompts); this thesis executes an 8-run subset (1 tool × 1 feature × 8 runs).

Readers evaluating this thesis should weight Chapter 6.2 (executed scope, $n = 5$ Cond A and $n = 3$ Cond B on one representative feature under one tool family) and Section 7.4 (planned replication across 5 tools, 3 scenarios, and 10 prompts) accordingly. The figure depicts the design-stage target architecture of the planned replication; only the 8-run subset is reported as empirical evidence in this thesis, not the full 150-artifact design-stage target.

1.4 Scope and Limitations

To keep the empirical rigor and feasibility of this research within the resource envelope of a single-author thesis, the scope is strictly defined by the following parameters and limitations:

1. **Framework scope:** Next.js App Router only. No comparison with Nuxt, Remix, or other meta-frameworks.
2. **AI assistants:** Cursor AI used for the pilot empirical case study. Broader AI assistant evaluation (GitHub Copilot, Claude Code) is future work.

3. **Project type:** Single representative Next.js project demonstrating SaaS patterns.
4. **Pilot study limitation:** The empirical case study is a worked-example executed against a total of 8 runs under a pre-registered adaptive schedule, distributed as $n = 5$ Cond A and $n = 3$ Cond B on a single representative feature, the `isEnabled(flagKey, userId?)` utility, which uses no service that SmartKit integrates. The pilot illustrates framework applicability and produces per-condition medians plus directional predictions (DP1–DP3), evaluated against the pre-registered verdict rule in Section 6.2. Cond B remains at $n = 3$, so the per-arm sample is still far below the $n \approx 30$ required for inferential validation (Section 7.4). Cross-tool replication, larger- N studies, and developer-participant validation remain future empirical work.

These boundaries are deliberate methodological choices. Fixing the framework, the AI assistant, and the project type removes confounding variance so that any measured difference between structured and unstructured conditions can be attributed to the SmartKit approach alone.

2. Background

Understanding the intersection of AI coding assistants and Next.js development requires grounding in three domains: the capabilities and limitations of current AI coding tools, the architectural patterns of the Next.js framework, and the principles of SaaS boilerplate design.

2.1 AI Coding Assistants

GitHub Copilot, Anthropic’s Claude, and Cursor AI all use transformer-based LLMs fine-tuned for code understanding and generation. GitHub Copilot provides inline code completions and entire function generation within supported editors (using OpenAI’s GPT series and Anthropic’s Claude series models, with specific model versions varying over time). Anthropic’s Claude offers conversational code assistance through the Claude Code CLI and IDE integrations. Cursor combines Claude with a custom codebase indexing system that enables context-aware suggestions across entire projects.

These assistants share common architectural elements. They process code through transformer-based models trained on large corpora of publicly available source code, learning statistical patterns that inform code generation. When producing completions, they consider surrounding code context, natural language comments, and project-wide information to generate relevant suggestions. Their effectiveness varies with programming language, task complexity, and code structure (Yetiştiren et al., 2023).

Research documents both strengths and limitations. Dakhel et al. found that AI assistance benefits expert developers who can filter suboptimal suggestions but may harm novice developers who lack the expertise to identify incorrect or inefficient code (Dakhel et al., 2023). Liu et al. documented that ChatGPT performs better on older problems (pre-2021) than newer ones, with a 48.14% advantage on the Accepted rate, though it struggles with complex algorithms and domain-specific code, frequently generating solutions with security vulnerabilities and inefficient implementations (Z. Liu et al., 2024).

The task taxonomy for AI suitability shows that boilerplate generation represents a strength area. Zhang et al. observed that developers primarily use Copilot for boilerplate generation, test writing, and code explanation rather than novel algorithmic development (Zhang et al., 2023). Pandey et al. confirmed this pattern, finding strong AI performance on template generation and documentation tasks but weak performance on complex logic and integration code (Pandey et al., 2024). These findings focus the study on boilerplate interaction as an appropriate domain for AI enhancement.

2.2 Next.js Framework Architecture

Next.js is a React meta-framework that provides server-side rendering, static site generation, and API capabilities within a unified application model. Version 13 introduced the App Router paradigm, fundamentally changing how React applications structure pages, layouts, and data fetching. Developers who learned the Pages Router encounter several App Router concepts that the older model did not surface, including Server/Client boundaries and file-system conventions for special files.

React Server Components represent a core innovation in Next.js App Router. These components execute exclusively on the server, which lets them access databases and sensitive operations directly without exposing logic to the client. The distinction between Server and Client components requires explicit marking through the `use client` directive, creating a boundary that developers must carefully manage. Incorrect placement of components across this boundary produces runtime errors and performance degradation.

The App Router organizes applications through a file-system-based routing mechanism. Each folder within the `app/` directory corresponds to a route segment, with `page.tsx` files defining the content for each route. Special files such as `layout.tsx`, `loading.tsx`, and `error.tsx` provide shared UI and error handling across route hierarchies. This convention-driven structure enables rapid scaffolding but requires adherence to specific naming and placement patterns.

TypeScript integration in Next.js emphasizes type safety throughout the application. The framework provides typed interfaces for page properties, API request and response objects, and data fetching results. Achieving full type coverage requires consistent use of TypeScript patterns including generics, utility types, and strict mode configuration. AI coding assistants may generate code that compiles but violates framework conventions, producing runtime errors or subtle bugs.

2.3 SaaS Boilerplate Patterns

Software-as-a-service applications share common requirements that boilerplate frameworks address: user authentication, database access patterns, multi-tenancy support, and deployment configurations. The economics of SaaS development favor approaches that reduce per-project initialization effort while maintaining code quality and consistency.

Krüger et al. established that platform reuse reduces maintenance effort but involves upfront investment in framework adoption and learning curve (Krüger & Berger, 2020). The trade-off explains the appeal of well-designed boilerplate templates that amortize initialization cost across multiple projects. Aleem et al. identified customization, scalability, multi-tenancy, security, integration, and fault tolerance as key factors in SaaS architecture decisions (Aleem et al., 2019).

Modern scaffolding approaches build on model-driven architecture principles. Inayatullah and Azam showed that automated code generation reduces boilerplate development effort for cross-platform applications (Inayatullah et al., 2019). Liu and Sra recently showed that LLMs have potential for automatically generating scaffolded user interfaces, reducing complexity for users in professional creative software (Y. Liu & Sra, 2025).

Meta-framework evaluation identifies architectural trade-offs across options. Prior work on framework adoption in software engineering has identified learnability, extensibility, and ease of migration as primary drivers of adoption, while framework lock-in and high configurability are commonly cited as major adoption risks (Storey et al., 2016). Next.js App Router emphasizes React Server Components as a first-class primitive, while maintaining full compatibility with Client Components for interactive UI. The hybrid model creates specific conventions that boilerplate frameworks should encode.

3. Related Work

This section surveys existing research on code generation benchmarks, empirical studies of AI-assisted development, and systematic literature reviews on AI for software engineering. The analysis identifies the specific gap this thesis addresses: the absence of framework-specific evaluation for AI coding assistants operating on Next.js boilerplate conventions.

3.1 Code Generation Benchmarks

The evaluation of AI coding assistants relies heavily on standardized benchmarks that measure code generation capability. Chen et al. introduced HumanEval as a foundational benchmark containing 164 hand-written programming problems with function signatures, docstrings, and unit tests (Chen et al., 2021). The benchmark evaluates models using Pass@1, Pass@10, and Pass@100 metrics. OpenAI’s Codex achieved 28.7% Pass@1 on this benchmark, establishing a baseline for subsequent model development.

HumanEval focuses exclusively on single-function Python code generation with algorithmic and reasoning requirements. The design prioritizes general programming capability over domain-specific or framework-aware generation. Yu et al. extended the benchmark concept with HumanEval Pro, adding self-invoking code generation tasks that require models to generate functions calling previously defined functions (Yu et al., 2025). Their work documented up to 30% performance degradation on self-invoking tasks compared to standalone tasks, indicating that current benchmarks underestimate real-world code generation complexity.

SWE-bench represents a substantial advance in benchmark realism by evaluating AI models on real GitHub issues from popular Python repositories (Jimenez et al., 2024). Each task requires understanding the issue description, locating relevant code, implementing a fix, and passing unit tests. Jimenez et al. found that state-of-the-art models resolved only a small fraction of issues — Claude 2 solved approximately 1.96% with BM25 retrieval (4.8% with oracle retrieval) and GPT-4 only 1.74% with oracle retrieval (0% with BM25) — and the gap between benchmark performance and practical software engineering capability is substantial.

Language-specific benchmarks identify further evaluation limitations. Petrukha et al. developed SwiftEval for Swift programming, finding that general-purpose benchmarks overestimated Swift code generation capability (Petrukha et al., 2025). Their language-specific benchmark documented 40–60% performance gaps not captured by cross-language evaluations.

No existing benchmark evaluates framework convention adherence, boilerplate structure quality, TypeScript or Next.js-specific patterns, or multi-file scaffolding tasks. The benchmarks measure whether AI assistants can solve algorithmic problems or fix bugs in existing codebases, not whether they can generate framework-compliant boilerplate from scratch.

Several extensions and alternatives warrant mention. Liu et al. developed HumanEval+ and MBPP+ as rigorous extensions of the original benchmarks, finding that pass rates drop often by 30% or more when evaluated against additional high-quality tests per problem (J. Liu et al., 2024). Zhuo et al. introduced BigCodeBench, which evaluates 1,140 fine-grained tasks combining diverse function calls with complex natural-language instructions (Zhuo et al., 2024). Hendrycks et al. proposed APPS, a 10,000-problem benchmark spanning introductory, interview, and competition-level coding challenges (Hendrycks et al., 2021). Jain

et al. addressed the critical contamination problem through LiveCodeBench, which sources problems from recent competitive programming contests published after model training cut-offs (Jain et al., 2024). Li et al. developed CodeReviewer for code review automation, an adjacent evaluation direction that assesses code quality dimensions relevant to AI-assisted development workflows (Li et al., 2022). Each of these benchmarks measures distinct aspects of code generation or code quality capability, yet none evaluate the framework-specific convention adherence that the SmartKit framework targets.

3.2 Empirical Studies on Developer Productivity

Several empirical studies examine how AI assistance affects real developer workflows. Peng et al. conducted the seminal controlled experiment on GitHub Copilot with 95 recruited developers implementing an HTTP server in JavaScript (Peng et al., 2023). Their methodology used a treatment-control design with matched tasks, finding the 55.8% completion time improvement cited earlier. Heterogeneous effects showed that less experienced developers benefited most from AI assistance.

The enterprise-level .NET study by Tariq et al. (SSRN, 2026; not peer-reviewed) extended this methodology to enterprise software development scenarios (Tariq et al., 2026). With 60 developers and 960 task observations across eight enterprise-oriented tasks, they measured task completion time, implementation errors, cyclomatic complexity, maintainability index, code smells, and NASA-TLX cognitive load scores. AI-assisted development reduced task completion time, errors, code smells, and cognitive workload while improving maintainability. The trade-off was that AI generated redundant abstractions and increased code verbosity, with effectiveness depending on developer expertise and continued oversight. Note that the Tariq et al. study focuses on .NET enterprise settings; its findings are cited as methodology precedent rather than as evidence about Next.js or AI-tool-specific effects.

Liu et al. assessed ChatGPT code generation quality across multiple languages and task types (Z. Liu et al., 2024). ChatGPT performs better on pre-2021 problems with a 48.14% advantage on Accepted rate, but its ability to directly fix erroneous code through multi-round dialogue is relatively weak. Generated code often contained security vulnerabilities and inefficient implementations requiring human verification, though ChatGPT could resolve over 89% of identified vulnerabilities through multi-round fixing. Yetiştiren et al. documented that Copilot generates functionally correct code for many problems but frequently produces solutions with higher cyclomatic complexity and lower readability than human-written solutions (Yetiştiren et al., 2022).

AlOmar et al. examined developer-ChatGPT refactoring conversations, finding that developers frequently needed multiple iterations to achieve satisfactory refactoring (AlOmar et al., 2024). AI often missed design-level improvements, and common anti-patterns emerged in developer-AI refactoring interactions. AI effectiveness therefore depends on the structure of the interaction and on the developer's ability to guide refinement, not only on the tool itself.

Hou et al. conducted a systematic literature review of 395 research articles on LLMs for software engineering from 2017–2024 (Hou et al., 2024). Code generation (35%) and code repair (25%) are the most studied application areas. Most studies use benchmark evaluation (60%) rather than human evaluation, and reported effectiveness varies widely by task type. The review identified research gaps in security, requirements engineering, and empirical studies. Framework-specific boilerplate evaluation would begin to address these gaps.

Russo et al. analyzed GenAI adoption complexity, identifying five key dimensions: technical, organizational, individual, process-related, and ethical (Russo et al., 2024). Technical capability was necessary but insufficient; human factors including trust, skill, and workflow integration were critical success factors. Evaluating framework-specific boilerplate tools therefore requires attention to how well they integrate with developer workflows and support skill development.

3.3 Evaluation Methodology

The refinement of AI-generated code has received attention as a complementary evaluation dimension. Liu et al. analyzed 4,066 ChatGPT-generated programs across 2,033 programming tasks in Java and Python (Y. Liu et al., 2024). Of these programs, 2,756 were deemed correct, 1,082 produced wrong outputs, and 177 contained compilation or runtime errors; 1,930 additional programs suffered from maintainability issues identified through static analysis. Their automated refinement pipeline using static-analysis feedback improved code quality by more than 20%. Evaluation frameworks should therefore measure both initial generation quality and the effort required for refinement.

Ouyang et al. studied the non-determinism of ChatGPT in code generation (Ouyang et al., 2025). Across 829 problems in three benchmarks, the ratio of coding tasks with zero equal test output across different requests ranged from 47.56% (HumanEval) to 75.76% (CodeContests). Setting the temperature to 0 does not guarantee determinism, despite being a common practice. Reliable assessment therefore requires multiple runs with confidence intervals rather than single-point measurements.

Beller et al. proposed the DAT framework for measuring software development productivity at Meta (Beller et al., 2026). The approach combines diff authoring time, survey-based metrics, and AI tool-specific productivity measures. The DAT framework models a path for evaluating AI-boilerplate interaction that balances quantitative performance metrics with qualitative developer experience assessment.

Nguyen-Duc et al. outlined a research agenda for GenAI in software engineering, emphasizing code generation quality, testing and verification, developer experience, education and training, ethics and security, and process and project management (Nguyen-Duc et al., 2025). The agenda explicitly calls for standardized evaluation and longitudinal studies, requirements that framework-specific evaluation frameworks can begin to satisfy.

Sauvola et al. projected four primary future scenarios for generative AI in software development, ranging from legacy integration to clean-slate and networked development, with software production shifting toward higher-level design and oversight roles as routine coding tasks become increasingly automated (Sauvola et al., 2024). Research on optimizing AI interaction patterns, including framework-specific boilerplate conventions that reduce the cognitive overhead of AI-assisted development, follows from this projection.

3.4 Research Gap Summary

The reviewed literature establishes that AI coding assistants improve developer productivity on repetitive tasks, that boilerplate generation represents an AI strength area, and that existing benchmarks measure algorithmic capability rather than framework convention adherence. To the author’s knowledge—given the structured-but-non-exhaustive literature search described in the next section—no prior work has specifically evaluated AI tool effectiveness

on Next.js App Router conventions, React Server Components patterns, TypeScript-first boilerplate structures, or full-stack framework-specific best practices. This gap is independently supported by Hou et al. (2024)’s systematic review of 395 LLM-SE articles, which does not list framework-specific convention adherence as a separately tracked evaluation category (Hou et al., 2024); the review’s documented research gaps (security, requirements engineering, empirical studies) likewise do not overlap with framework-specific boilerplate evaluation. This thesis proposes a Next.js boilerplate framework engineered for AI interaction and an evaluation methodology that measures framework-specific quality dimensions, reporting both as design-stage contributions.

3.5 Literature Search Methodology

The literature reviewed in this chapter was identified through a structured search process. Primary databases searched include ACM Digital Library, IEEE Xplore, arXiv, and Google Scholar. Search terms combined AI coding assistant terminology (“LLM code generation,” “Copilot evaluation,” “ChatGPT code quality”) with framework-specific terminology (“React Server Components,” “Next.js,” “TypeScript boilerplate”) and benchmark terminology (“code generation benchmark,” “framework-specific evaluation”). Inclusion criteria prioritized peer-reviewed publications from 2020 onward, with seminal pre-2020 works (e.g., HumanEval, Codex) included for foundational context. The 35 references in the bibliography are supplemented by the systematic literature review of Hou et al. (Hou et al., 2024) which provides independent coverage of 395 articles in the LLM-for-SE space. The author acknowledges that a fully PRISMA-compliant systematic review with explicit screening protocol is beyond single-author-thesis scope; the search described here is structured but not exhaustive, and additional framework-specific evaluation work may exist outside the search scope.

3.6 Alternative Approaches to AI-Codebase Alignment

The structural approach taken by SmartKit is one of several strategies that researchers and practitioners have explored to improve AI-codebase alignment. This section briefly reviews these alternatives and justifies why the structural approach was selected for this thesis.

Prompting strategies represent the most lightweight alternative. Enhanced system prompts, few-shot examples, and retrieval-augmented generation (RAG) can improve AI output quality without modifying project structure. Prompting approaches suffer from context window limitations, require repeated re-instruction across sessions, and cannot enforce consistency across a codebase without external governance. Hou et al. (2024) note in their systematic review that most LLM-for-SE studies use benchmark evaluation (60%) rather than human evaluation, and reported effectiveness varies widely by task type; this leaves open the question of how prompting-only approaches hold up outside benchmark settings (Hou et al., 2024). Prompting alone cannot address the structural convention-adherence gap the review identifies.

Enhanced documentation such as thorough READMEs, inline comments, and architecture decision records (ADRs) can guide AI behavior. Documentation requires human maintenance, is easily ignored by AI systems generating new code, and does not provide machine-readable enforcement mechanisms.

Linter and formatter configurations (ESLint, Prettier) can catch some convention violations but operate post-generation rather than preventing them. These tools validate

style, not semantics; they cannot enforce feature boundaries or Server/Client component conventions that require architectural awareness.

The **structural approach** taken by SmartKit addresses these limitations by encoding conventions at the project structure level. Conventions become discoverable by AI systems through file location and naming patterns, are enforced through project structure rather than developer vigilance, and provide reusable templates that scale across features. The structural approach works alongside prompting and documentation, not as a replacement.

Caveat on the alternative-approaches dismissals. The dismissals of prompting, enhanced documentation, and linter-based enforcement above are *design-stage reasoning* based on prior literature, not empirical comparisons. The pilot’s Condition A advantage is the *combined* effect of multiple overlapping signals—the AGENTS.md instructions, the visible example features (features/auth/, features/billing/), and the existing linter/Zod configuration—and the executed pilot does not isolate “structural” conventions from the prompting, documentation, and tooling layers that operate alongside them. A controlled comparison of structural vs. prompting vs. documentation vs. linter-only conditions is left as future work (Section 7.4); the present thesis reports only the bundled effect of the SmartKit convention envelope.

Table 3.1 positions SmartKit qualitatively against five established Next.js starter templates on seven dimensions relevant to AI-assisted boilerplate generation. The comparison is descriptive rather than empirical: cells summarise publicly documented features of each template, not measured outcomes, and no performance claim should be drawn from them.

Table 3.1: Qualitative positioning of SmartKit against established Next.js starter templates. Cells describe documented features, not measured performance; cell-level claims were transcribed from each template’s own documentation.

	SmartKit	create-t3-app	Next.js official	Vercel templates	shadcn/ui	bulletproof-react
Feature-based layout	Yes	Partial	No	No	No	Yes
Server/Client convention	'use client'	tRPC types	Manual	Manual	Manual	Manual
ORM included	Drizzle	Prisma	None	None	Drizzle	Prisma
Auth library	better-auth	NextAuth.js	None	NextAuth.js	Auth.js	Context API
TypeScript-first	Yes	Yes	Optional	Optional	Yes	Yes
Testing scaffold	Vitest	Vitest	None	None	Vitest	Vitest
AGENTS.md	Yes	No	No	No	No	No

¹

The qualitative positioning in Table 3.1 suggests a structural approach to convention encoding in AI-assisted development. While most frameworks provide partial feature-based layout or manual Server/Client conventions, SmartKit includes AGENTS.md as a first-class project artifact—enabling persistent, non-negotiable convention encoding directly readable

¹create-t3-app: <https://create.t3.gg/>; Next.js official: <https://nextjs.org/docs/app/getting-started>; Vercel templates: <https://vercel.com/templates>; shadcn/ui: <https://ui.shadcn.com/>; bulletproof-react: <https://github.com/alan2207/bulletproof-react>. SmartKit claims trace to the source repository accompanying this thesis.

by AI coding assistants. This positioning reflects the design intent of SmartKit; empirical validation (developer-participant comparison against feature-based-only or documentation-only conventions) is documented as future work in §7.4.

4. Proposed Solution

This section presents the design of SmartKit, a Next.js boilerplate framework engineered to optimize interaction with AI coding assistants. The design synthesizes empirical findings from the literature with domain-specific analysis of Next.js conventions to produce a codebase structure that improves AI comprehension and output quality.

4.1 Design Rationale

Zhang et al. found that structured, well-documented codebases improve AI coding assistant performance (Zhang et al., 2023). Barke et al. showed that grounded prompting, providing relevant code context, improves AI output quality (Barke et al., 2022). The SmartKit design applies these findings through concrete architectural decisions: encode conventions explicitly so that AI systems can discover them, group related code so that context stays narrow, and define type-safe contracts so that data shapes remain explicit.

These principles translate into four concrete design choices, each chosen over an alternative.

Feature-based modularity over layer-based organisation. Many Next.js projects organise code by technical layer (`components/`, `lib/`, `hooks/`) or by page. Layer-based organisation creates wide import graphs: a single component may transitively import from dozens of files, so an AI system generating a new component receives a large, undifferentiated context. Feature-based modularity confines all code for one feature (schemas, actions, components, emails, types) within a single directory, giving the AI a narrow, self-contained context window. The trade-off is that cross-cutting concerns (shared utilities, types, or components) require deliberate extraction to a shared layer; this overhead is acceptable for SaaS boilerplate where the feature boundary is the primary decomposition axis.

Drizzle ORM over Prisma. Prisma uses a schema-first DSL (`.prisma` file) that generates a client; Drizzle uses a TypeScript-first schema directly in `.ts` files, co-located with the application code. The TypeScript-native approach means Drizzle schemas are importable as regular modules without a generated-client intermediary, and schema changes propagate through TypeScript’s own type inference without a separate code-generation step. For AI-assisted code generation, co-located TypeScript schemas are more likely to be surfaced in context than an external DSL file; the AI can read and extend them using the same language model it uses for the rest of the codebase. The trade-off is that Drizzle requires more explicit SQL awareness from the developer; Prisma’s generated client abstracts SQL differences.

Explicit Server/Client boundaries over tRPC-based type sharing. tRPC provides end-to-end type inference across the Server/Client boundary by sharing a single TypeScript interface definition. This eliminates a class of type errors but requires a separate router definition and does not enforce the architectural boundary at the file level. SmartKit instead uses Next.js’s native `'use~client'` directive for Client Components and the `server-only` import for server-only utilities, supplemented by Zod schemas shared across the boundary (see below). The native mechanism is framework-default rather than library-specific, which means the conventions survive beyond the SmartKit codebase and transfer to plain Next.js projects; tRPC conventions do not. The trade-off is that the native mechanism requires explicit developer discipline; tRPC enforces the boundary automatically through its type system.

Zod schemas as the single source of truth across the Server/Client boundary. React Server Components and Client Components communicate through typed props, but there is no built-in mechanism to ensure that the type on the server matches the type on the client. SmartKit uses Zod schemas, defined once in `features/<name>/types/`, parsed at the server action boundary, and reused on the client via `z.infer`. This means a single schema definition is the source of truth for both runtime validation (server action) and TypeScript type inference (client component props), and the AI can read and extend the schema using the same Zod syntax it encounters in existing features. The trade-off is that Zod adds a runtime dependency and a learning curve for developers unfamiliar with schema-first validation.

4.2 Feature-Based Modular Architecture

Many Next.js projects organize code by technical layer (`components/`, `lib/`, `hooks/`) or by page structure. SmartKit instead adopts feature-based modularity, where each feature contains all code necessary for its functionality. An authentication feature, for instance, includes database schemas, API routes, React components, form validation schemas, and email templates within a single directory.

This organization improves AI comprehension by narrowing the search space for relevant context when AI generates code for a specific feature. The scope of changes becomes explicit, which reduces the risk of AI-generated code violating feature encapsulation. The organization also creates natural boundaries for testing and deployment that align with developer mental models.

The feature directory structure follows a consistent pattern across all features:

```
1  src/features/{feature-name}/  
2  +-- components/           # React components  
3  +-- lib/                  # Feature-specific utilities  
4  +-- types/                # TypeScript type definitions  
5  +-- emails/               # Email templates  
6  +-- index.ts              # Public API barrel
```

Listing 4.1. *Feature directory layout convention used across SmartKit.*

This structure allows AI systems to predict where to place new code within a feature, reducing the cognitive overhead of project organization.

4.3 Server/Client Boundary Conventions

React Server Components introduce an explicit boundary between server-only and client-accessible code. SmartKit uses Next.js's native Server/Client boundary mechanism through the `'use~client'` directive for Client Components and the `server-only` import for server-only utilities. The pattern extends Next.js's convention-based routing to component architecture.

Server/Client boundaries affect data flow design. Server Components can access databases and environment variables directly, while Client Components must receive data through props or server actions. AI systems can invoke type-safe server actions from Client Components, which keeps data flow consistent as new components are generated.

Listing 4.2 shows a representative excerpt from the SmartKit `AGENTS.md` file, which encodes the conventions that Cursor AI reads when operating inside the project. The file

defines project-level rules that take precedence over instructions found in code comments or tool outputs, and includes specific conventions for file placement, Server/Client boundaries, and schema sharing.

```
1 # SmartKit Boilerplate -- Code Conventions
2
3 Rules for AI coding assistants working inside smartkit/.
4
5 ## Project Shape
6 - Next.js 16 App Router, TypeScript 5.7 strict mode
7 - Feature-based layout: all feature code lives under src/
  features/{name}/
8 - Zod 4 for schema validation, Drizzle ORM for the database,
  Biome for lint/format
9 - Vitest + Testing Library + Happy DOM for tests
10 - Authentication via Better Auth; email via Resend + React
    Email templates
11
12 ## Feature Directories
13 - Place all feature code under src/features/{name}/
14 - Default subdirectories: components/, hooks/, actions/, lib/,
    types/, emails/
15 - Strong default, not enforcement: single-file features may
    live at the feature root
16
17 ## Server/Client Boundaries
18 - Client Components: 'use client' directive at the top of the
    file
19 - Server-only utilities: server-only import to prevent client
    bundling
20 - Do not use .server.ts / .client.tsx filename conventions
21
22 ## Schema Sharing
23 - Zod schemas are defined once in the feature types/ directory
24 - The same schemas validate payloads on the client before
    submission and on
25   the server inside route handlers and server actions
26 - Keep type definitions explicit; avoid any types
```

Listing 4.2. *Representative excerpt from SmartKit's AGENTS.md file.*

4.4 Database Schema Patterns

SmartKit uses Drizzle ORM with PostgreSQL to provide type-safe database access throughout the application. The schema design follows conventions that AI systems can learn and reproduce. Each table schema includes columns with explicit types, relations defined with foreign keys, and indexes specified for query optimization.

The Drizzle schema pattern enables AI-generated code to access the database through type-safe query builders. AI systems can examine existing schema patterns and generate consistent queries that maintain type safety across the application. Ad-hoc database access

that bypasses type checking or violates query conventions is discouraged.

Schema files for Better Auth's shared tables (`user`, `session`, `account`, `verification`) are all implemented in `src/database/schema/`, an infrastructure layer outside the Feature-Sliced Design (FSD) hierarchy, alongside the billing-specific tables (`billing`, `payment\_transactions`) used by `features/billing/`. The four Better Auth tables cover the user identity, the `session` and `account` records used for sign-in flows, and the `verification` table for email-verification challenges; relations between them are declared in `src/database/schema/relations.ts`. All Drizzle schema definitions are kept in this single directory so that `drizzle-kit` can scan them reliably and all features can import from a common location. The placement keeps database patterns centralized while remaining directly importable from any feature module.

4.5 Type-Safe API Contracts

API routes in SmartKit use Zod schemas for request and response validation. These schemas serve dual purposes: runtime validation of incoming requests and TypeScript type inference for handler code. AI systems can examine Zod schemas to understand API contracts, generating code that respects type constraints without requiring explicit type annotations.

The shared schema pattern enables consistent validation across frontend form components and backend API handlers. The same Zod schemas validate both client-submitted forms and server-handled requests. The match between frontend and backend validation reduces the gap between AI-generated client-side and server-side code, producing more coherent implementations.

5. Implementation

This section describes the technical implementation of the SmartKit framework using Next.js 16.2.12 and associated technologies. The implementation shows how the design principles translate into working code.¹

5.1 Technology Stack

5.1.1 Stack Selection Rationale

Each component of the stack was chosen because it reinforces one of the three evaluation dimensions rather than for its own sake. The selection rests on three guiding principles:

1. **App Router-first.** Next.js 16's App Router is the architectural surface that Structural Integrity measures directly: Server/Client boundaries, file-system routing, and the 'use~client'/server-only primitives are framework defaults rather than library conventions, which means the rules encoded in the rubric transfer to plain Next.js projects rather than coupling them to SmartKit.
2. **Type-safe contracts at the boundary.** TypeScript 5.7 strict mode provides the compile-time signal that Type & Contract Safety reads from `tsc--noEmit`, while Zod 4 schemas colocated in `features/<name>/types/` are parsed at the Server Action boundary and reused on the client via `z.infer`. A single schema is the source of truth on both sides of the Server/Client boundary that Coverage & Completeness checks for, so the AI tooling reads and extends the same Zod syntax it encounters in existing features.
3. **Server/Client boundary as a first-class primitive.** The framework's runtime failures (event handlers crossing the boundary, server-only imports leaking to the client) are the failure mode the thesis targets, so the chosen tools must surface those failures naturally: Server Components fetch data directly through Drizzle, Server Actions in `features/<name>/actions/` call `revalidatePath` after writes, and Client Components receive typed props without an intermediate client-side data layer. Adopting a tRPC-style shared-router alternative would have hidden the boundary inside library types and removed the very signal the Structural Integrity rubric is designed to read.

5.1.2 Dependency Version Pins

Table 5.1 lists the canonical versions of the libraries that comprise the stack. Versions are pinned to the values declared in `smartkit/package.json` at the snapshot used for the pilot evaluation (Section 6); reproducing the rubric against a newer release is out of scope and would require re-scoring the rubric cells.

¹The pilot evaluation (Chapter 6) exercises a single representative feature — the `isEnabled(flagKey, userId?)` flag utility — that touches none of SmartKit's integrated services. The billing (Section 5.4), authentication, and email integrations documented below are not exercised by the pilot; they are documented here as part of SmartKit's second role — a usable SaaS boilerplate — independent of the evaluation framework. See Section 6.1.1 for the relationship between the integrations and the pilot feature.

Table 5.1: Canonical dependency pins for SmartKit at the pilot-evaluation snapshot. Versions match `smartkit/package.json`; `better-auth` lists the latest 1.6.x line as of the snapshot.

Dependency	Version
Next.js	16.2.12
React / React DOM	18.3.1
TypeScript	5.7.3
Drizzle ORM	0.45.2
Zod	4.4.3
Biome	2.5.6
Tailwind CSS	4.3.3
Resend	6.18.1
React Email	6.9.1
better-auth	1.6.x
Vitest	4.1.x

Next.js 16.2.12 supplies the App Router architecture; Drizzle ORM 0.45.2 (PostgreSQL) and Zod 4.4.3 jointly enforce the type-safe-contract principle from §5.1.1. The remaining tooling is chosen for consistency rather than for a direct role in the rubric: Tailwind CSS 4.3.3 (CSS-first configuration) for styling, Biome 2.5.6 as a single linter/formatter in place of separate ESLint and Prettier configurations, Resend 6.18.1 with React Email 6.9.1 for transactional email templates, and SePay as the payment gateway integrated through `features/billing/` (HMAC signature verification + idempotent webhook handling; described in detail in §5.4). `@better-auth/drizzle-adapter` integrates authentication with the database layer. The relationship of these four to the pilot evaluation is summarised in the footnote at the start of this chapter; their individual roles are detailed where each is discussed below.

5.2 Project Structure

The SmartKit project structure implements the feature-based architecture described in Section 4:

```

1 smartkit/
2 +-- src/
3 |   +-- app/           # Next.js App Router pages
4 |   +-- features/      # Feature-based modules
5 |   |   +-- auth/      # Authentication feature
6 |   |   |   +-- components/ # React components
7 |   |   |   +-- emails/    # Email templates
8 |   |   |   +-- lib/      # Auth-specific helpers
9 |   |   |   +-- types/    # Auth type definitions
10 |   |   |   +-- index.ts  # Public API barrel
11 |   +-- database/      # Drizzle schema, migrations, client
12 |   |   +-- schema/    # user, session, account,
13 |   |   |   verification (Better Auth tables) + billing +
14 |   |   |   payment_transactions
15 |   |   +-- migrations/ # Drizzle migration files
16 |   |   +-- db.ts       # Drizzle client
17 |   +-- lib/           # Shared utilities
18 |   +-- config/        # Site and environment config

```

Listing 5.1. *Top-level directory structure of the SmartKit instantiation.*

The `app/` directory is placed at `src/app/` because that is the path Next.js 16's App Router reads as the routing surface. Pages and layouts live here, not feature code, so the route composition that the Structural Integrity rubric inspects (server-only imports, 'use~client' directives, file-system routing) is read directly from this directory by the framework itself rather than from a custom layout the AI would have to learn. Each route imports feature modules from `src/features/` but never the other way around, which is what keeps the page-vs-feature separation enforceable by static analysis: a feature module cannot accidentally import from a route, because that would invert the dependency direction the App Router expects.

The `database/` directory is placed at `src/database/` because `drizzle-kit` reads schemas from a single configured path; co-locating every table in `database/schema/` means a migration scan touches one root rather than chasing feature-local schema files. Hosting the four Better Auth tables (`user.ts`, `session.ts`, `account.ts`, `verification.ts`), the relations graph in `relations.ts`, and the billing tables (`billing.ts` with `payment\_transactions`) together lets any feature module import shared row types (`database/schema/index.ts`) without depending on a sibling feature's directory. The `database/db.ts` client also sits here so that the `pg` pool, the `env.DATABASE_URL` read, and the schema registration live in one file — the runtime surface that Type & Contract Safety checks via `tsc--noEmit` stays minimal and central.

The `lib/` directory is placed at `src/lib/` because the utilities inside (the `cn` class-name helper in `utils.ts`, the typed `env.ts` reader, any future cross-feature helper) do not belong to a single feature. Putting `cn` inside `src/features/auth/lib/` would force the billing feature to import from `auth` just to merge class names, which inverts the feature-isolation rule `AGENTS.md` sets and creates a spurious cross-feature dependency for a generic helper. The directory therefore functions as a third tier, sitting above `src/features/<name>/` in the import graph, for code that every feature may need but no feature owns. The `env.ts` reader belongs here for the same reason: the database client, the auth handler, and the billing webhook all read environment variables, so the reader sits at a level that all three can import without forming a cycle.

The `features/flags/` directory sits at the repository top level rather than inside `src/`. This placement is deliberate: the `AGENTS.md` rule that new feature code should live under a feature-named directory does not enforce a `src/` prefix for the pilot's representative feature, and keeping `features/flags/` at the top lets the AI encounter it immediately when scanning the repo root. The pilot's baseline-bootstrap script adds this directory so that the framework has a non-trivial integration target for the Coverage dimension without requiring a hand-written feature.

Internally, `features/flags/` follows the same conventions as `src/features/auth/`: `lib/` holds the `isEnabled` implementation, `types/` holds the Zod-derived `FlagKey` schema, and `index.ts` is the public API barrel. The unit test sits at `src/tests/features/flags/is-enabled.test.ts`, inside `src/`, in line with the centralized-test convention in `AGENTS.md`. Its presence is what makes the pilot executable against the framework without having to first create a feature by hand.

5.3 Key Implementation Patterns

The Server/Client boundary convention introduced in Section 4 is enforced consistently at implementation level: every generated file is checked for the `'use~client'` directive or `server-only` import appropriate to its role, which is what `score-flag.ts` inspects for the Structural Integrity dimension.

Database queries use Drizzle's type-safe query builder with explicit schema references. Generated code can use TypeScript's type inference to maintain consistency with existing queries. The query patterns include pagination, filtering, and relation loading that AI systems can adapt for new features.

API routes use Next.js Route Handlers with Zod schema validation. Request handlers receive parsed and validated data through a consistent interface, while response formatting uses a standard structure that includes status codes, data payloads, and error messages.

5.4 Billing and SePay Integration

The `features/billing/` feature implements a payment and order-management flow backed by SePay, a Vietnamese payment gateway. SePay webhook handling uses HMAC-SHA256 signature verification (`signature.ts`) and idempotent processing via a payment-transactions dedup key (`payment-transactions.ts`), so that repeated delivery attempts under transient failures produce a no-op rather than duplicate side effects. This feature serves as a second-tier pattern source for AI code generation in Condition A: its feature-based directory layout, server-action conventions, and Zod schema patterns are visible to the AI alongside `features/auth/`, giving the AI additional convention anchors beyond the explicit `AGENTS.md` rules. The billing feature's relation to the pilot evaluation is summarised in the scope note at the start of this chapter (and detailed further in §6.1.1); the SePay integration is described here as part of SmartKit's second role as a usable SaaS boilerplate and has not been independently audited for security, load, or test coverage.

6. Evaluation

This section presents the proposed evaluation framework and pilot feasibility study for assessing AI coding assistant performance on Next.js boilerplate generation tasks. The framework design is accompanied by a pilot evaluation protocol that exercises the framework on a representative feature.

6.1 Evaluation Framework

The framework design is accompanied by a pilot evaluation protocol that exercises the framework on a representative feature. §6.2 reports the locked protocol used to produce the empirical data, and §6.3 reports the data itself.

6.1.1 Three Dimensions

The evaluation uses a multi-dimensional quality framework that captures different aspects of boilerplate quality. The framework extends existing benchmark methodology to address framework-specific concerns that existing evaluations overlook.

Recent systematic research has identified a fundamental gap in how AI coding benchmarks measure capability. Hou et al. (2024), reviewing 395 LLM-SE articles, found that code generation (35%) and code repair (25%) dominate the literature, that most studies rely on benchmark evaluation (60%) rather than human evaluation, and that framework-specific convention adherence is not a separately tracked evaluation category (Hou et al., 2024). SmartKit’s contribution is to instantiate this missing measurement dimension for Next.js App Router.

The three dimensions capture distinct, non-overlapping aspects of boilerplate quality that affect real-world usability. Rather than measuring only output correctness, the framework assesses whether generated code can be integrated into production codebases without convention violations or type safety issues. Each dimension maps to an automated measurement approach and corresponds to a specific failure mode observed in AI-generated Next.js code.

Unit of analysis. Scoring is performed per generated file: each file receives one score per dimension, and feature-level scores are computed as the median across constituent files. The median is used rather than the mean because individual files within a feature may exhibit extreme outlier scores due to edge-case generation failures (e.g., a single malformed file), and the median is robust to such outliers.

Structural Integrity assesses whether generated code follows Next.js conventions for file location, naming, Server/Client component boundaries, and App Router patterns. Automated checking uses the `score-flag.ts` script (file-placement checks, `server-only` import detection, `'use client'` directive checks, barrel-file verification) specific to Next.js structural conventions. The dimension captures violations such as incorrect file placement in the `features/` directory, missing `'use~client'` directives on client components, and non-standard naming patterns that break routing.

Type & Contract Safety captures two related but distinct quality signals, treated as sub-scores that combine into a single Type & Contract Safety score on the 0–5 rubric. *Type Safety sub-score* focuses on compile-time checks: `tsc--noEmit` error counts and

`@typescript-eslint/no-explicit-any` occurrences signal whether generated code introduces type regressions into the codebase. *Contract Safety sub-score* focuses on runtime validation: the proportion of API route handlers that include Zod schemas for request/response validation indicates whether the generated feature is schema-validated at the API boundary (input parsing and type narrowing enforced at runtime via Zod), not production-deployment-ready without additional hardening such as security review and load testing (see Section 7.2). Sub-score combination rule: the Type & Contract Safety dimension score equals the lower of (Type Safety sub-score, Contract Safety sub-score) on the 0–5 scale, reflecting that a single weak sub-signal is not offset by a strong other. This combination rule is an inferred design choice (alternative rules such as arithmetic mean or weighted product could be substituted without changing the underlying sub-scores); the chosen lower-of rule privileges robustness against hidden weakness in either sub-dimension.

Coverage & Completeness evaluates whether generated boilerplate includes all necessary artifacts for a functional feature. Table 6.1 shows the reference checklist for the `isEnabled` feature flag utility used in the pilot. Completeness is measured by enumerating generated files against the checklist and computing a coverage ratio (files present divided by required files). Missing artifacts indicate incomplete boilerplate that requires additional developer effort to make functional.

Relationship between SmartKit Integrations and the Pilot Feature

SmartKit instantiates the framework conventions across multiple feature areas, including billing (SePay integration), authentication (Better Auth), and email (Resend). These integrations exist because the artifact is a usable SaaS boilerplate, not because the evaluation framework requires them. The pilot deliberately selects a feature—the feature-flag check utility—that uses *none* of the integrated services, in order to isolate the effect of convention encoding from the effect of having integrated services available. If the pilot used a feature that integrates SePay or Resend, any observed difference between Condition A and Condition B would conflate convention encoding with integration availability. The integrations remain first-class in the artifact; their relation to the pilot evaluation is summarised in the scope note at the start of this chapter. This separation is what allows the framework to remain generalisable beyond SmartKit: a developer using a different stack can encode the same conventions without SePay, Resend, or Better Auth, and the framework still measures the same three dimensions on any generated feature.

A second, related point concerns the role of the integrations during the AI session. In Condition A, the AI sees the full SmartKit file tree, including `features/billing/` (SePay HMAC, idempotency), `features/auth/` (Better Auth schemas and server actions), and the Resend email templates under `features/auth/emails/`. These files belong to SmartKit’s second role as a usable SaaS boilerplate (see the scope note at the start of this chapter); they encode the SmartKit convention envelope — feature-based folder structure, colocated server actions, Zod schemas in `types/`, `server-only` boundaries, and `revalidatePath` after mutations. The AI in Condition A pattern-matches this envelope when generating the pilot feature, even though the pilot feature itself does not call any of the integrated services. In Condition B, none of these pattern sources are available, so the AI must invent both the utility and its surrounding convention. The score difference between conditions therefore measures the convention envelope, not the integration surface. The convention envelope is what the framework is designed to capture; the integration surface is what makes SmartKit a usable SaaS boilerplate. Both are required for the thesis, but they play distinct roles. The integration surface contributes to the second thesis contribution—the SmartKit artifact as a working SaaS

boilerplate—rather than to the pilot; the pilot is the instrument that exercises the framework on AI-generated code, while the integrations are the evidence that the framework conventions are usable in production code.

The standardised prompt’s first resolution-order rule—“Per-user DB override, only if `userId` is passed”—is implemented in the pilot as a stubbed path: the generated `readUserOverride` helper returns `undefined` without querying any database layer, so the pilot feature genuinely touches neither Drizzle nor any external persistence. The stub keeps the resolution order observable in tests while leaving full DB-backed override as future work; this is consistent with the feature’s design choice to avoid integrating any SmartKit service, including Drizzle, into the pilot scope. The relationship between this design choice and the Coverage rubric’s level definitions (rubric A.5.3) is revisited in the pilot discussion (§6.2 and §7.2), which explains the structural constraint the design imposes on the Coverage dimension; the corrected Coverage cells in Table 6.8 and the correction footnote immediately following that table record the resulting scores.

Table 6.1: Reference coverage checklist for the `isEnabled` feature flag utility.

#	Artifact	Expected path
1	Core utility function	<code>features/flags/lib/is-enabled.ts</code>
2	Zod schema and FlagKey enum	<code>features/flags/types/flags.ts</code>
3	Unit test file	<code>src/tests/features/flags/is-enabled.test.ts</code>
4	Public API barrel	<code>features/flags/index.ts</code>
5	Cache implementation	Embedded in <code>is-enabled.ts</code> (5-minute in-memory cache)

Note: The `isEnabled(flagKey, userId?)` utility resolves feature flags with three override layers (per-user DB override, per-flag environment variable, default value) and caches results for five minutes. This feature was chosen for the pilot because it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), eliminating integration-pattern confounding.

6.1.2 Design Rationale and Trade-offs

The evaluation framework design involves several deliberate choices that balance scope with feasibility.

Dimension selection. The three dimensions were designed to address the gap between output-oriented benchmarks (algorithmic correctness) and the framework-specific quality dimension that existing benchmarks do not measure. Each SmartKit dimension targets a distinct failure mode observed in AI-generated Next.js code: structural violations (file placement, Server/Client boundaries), type safety gaps, and missing boilerplate artifacts.

Automated over manual evaluation. The framework prioritizes automated metrics to enable reproducibility and scale. Expert review provides richer qualitative assessment, but automated checking ensures consistent evaluation across the large number of code samples required for meaningful comparison. Where qualitative judgment is necessary (e.g., structure quality assessment), LLM-as-judge with majority voting is a future option that was not exercised in the executed pilot; the pilot uses single-primary-rater manual scoring (deferred IRR; see §7.2).

6-level rubric across dimensions. Each of the three dimensions is scored on a 0–5 rubric (six integer levels). The per-dimension cell descriptions are detailed in §6.2.4; the rough mapping is: 0 = missing artefact, 1 = severe violation, 2 = partial (in the right neighbourhood

but wrong sub-detail), 3 = acceptable, 4 = good (level 3 + a specific bonus criterion, e.g. server-only for Structural), 5 = excellent (level 4 + a second bonus criterion, e.g. barrel re-export). The exact level descriptions are dimension-specific and were inferred from related literature on code quality rubrics (the 0–5 scale mirrors industry convention for code-review rubrics); per-level cutoffs have not been calibrated against developer-experience data within this thesis.

Single-framework focus. The evaluation focuses exclusively on Next.js to maintain depth over breadth. Future work should extend the framework to Nuxt and Remix to assess generalizability of the approach.

Confound control between conditions. A central threat to the validity of the pilot results is confounding: the structured SmartKit condition and the unstructured condition differ *primarily* in convention encoding, but they also differ in second-order ways that the executed pilot does not fully control. The most important residual confound is that the SmartKit repository exposes several example features (`features/auth/`, `features/billing/`, `features/auth/emails/`) that the AI can use as pattern-matching anchors beyond the explicit AGENTS.md instructions. To isolate SmartKit’s contribution from this confound, the pilot compared it against an unstructured Next.js baseline sharing the same dependency surface as SmartKit (Next.js 16.2.12, TypeScript 5.7.3, Tailwind CSS 4.3.3, Drizzle ORM 0.45.2 with PostgreSQL, Zod 4.4.3, Biome 2.5.6, Resend 6.18.1, and better-auth 1.6.x, plus the `src/database/schema/` directory inherited from SmartKit) but with no feature-based `src/features/` layout and no AGENTS.md. The exact dependency surface used to build the baseline is recorded in §6.2.1 (Condition B paragraph) for replication; the `smartkit-baseline/package.json` shipped alongside this thesis is the canonical record of resolved versions and lock state. The exact `create-next-app` CLI invocation and the post-creation edits (customised `.env.example`, removal of AGENTS.md and `.cursor/rules/`) are kept in the baseline repository’s git history and are not duplicated in the bound thesis. The pilot feature (`isEnabled(flagKey, userId?)`) was chosen for the same reason: it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema), so the AI cannot copy a pattern from the example features. This design follows established experimental practice for fair comparison (Peng et al., 2023). This section refers to the baseline as the “convention-naïve scaffold” to avoid confusion with the SmartKit repository directory itself; the build script that produces it is preserved alongside the thesis artefacts (§6.2).

The control artifact serves as the control condition, providing a baseline that isolates the effect of SmartKit’s convention encoding from the effect of the underlying technology stack.

6.1.3 Evaluation Metrics

Quantitative metrics derive from automated analysis of generated code. *Structural Integrity* uses Biome custom rules to check Next.js-specific file placement, naming, and Server/Client boundary conventions. *Type & Contract Safety* uses `tsc --noEmit` output to identify type violations and custom Biome rules for *any*-type detection and Zod coverage. *Coverage & Completeness* uses file enumeration against a reference checklist per feature type.

Qualitative metrics derive from expert review of generated code. Reviewers assess whether generated code follows Next.js best practices, whether naming conventions are consistent, and whether the overall implementation aligns with framework expectations. The expert review layer follows an inter-rater reliability (IRR) protocol: two independent raters score the same generated artifacts, with a target Cohen’s $\kappa \geq 0.7$. The automated scoring

layer (Biome, `tsc--noEmit`) is deterministic and requires no IRR. The IRR protocol is documented here as a design; execution of the IRR pass is identified as future work pending availability of a second rater.

Productivity metrics such as diff authoring time and developer-reported productivity (Beller et al., 2026) are not collected in the current pilot and remain future work; the executed pilot reports only per-run time-to-correct (ttc, in minutes) as a coarse timing signal. Task completion time, cyclomatic complexity, maintainability index, and code smell density are documented as design-stage metrics for a future developer-participant study.

6.2 Pilot Methodology

The pilot evaluates whether providing architectural convention documentation improves AI coding assistant adherence to Next.js conventions and type safety. The independent variable is the presence of SmartKit’s `AGENTS.md`, `.cursor/rules/`, and `src/features/` convention documentation. The dependent variables are the three rubric scores (Structural Integrity, Type & Contract Safety, Coverage & Completeness) on a 0–5 scale.

The pilot feature (`isEnabled(flagKey, userId?)`) was selected because it uses no service that SmartKit integrates (no Resend, Better Auth, SePay, or Drizzle dependency beyond the schema). This choice keeps the comparison clean: by using a feature that touches none of SmartKit’s integrated services, the comparison isolates convention-encoding effects rather than conflating them with service-specific patterns. The evaluation scenario measures AI-generated boilerplate quality directly: AI systems receive task descriptions, and the generated code is measured against the framework dimensions. The current pilot focuses on Cursor AI as the primary case study, consistent with the scope defined in Section 1. All evaluations use multiple runs to address non-determinism concerns raised by Ouyang et al. (Ouyang et al., 2025); the Cursor AI tool does not allow user-settable temperature in Auto mode (the tool snapshot used in the pilot), so multi-run evaluation substitutes for temperature variation as the determinism-handling mechanism. Results report per-condition medians and ranges, plus Wilson 95% confidence intervals on the binary pass rate ($\geq 3/5$).

6.2.1 Conditions

Table 6.2 summarises the two conditions used in the pilot evaluation. Condition A is SmartKit at full setup; Condition B is a convention-naïve Next.js scaffold produced by `create-next-app` with the same dependency surface as SmartKit but no FSD layout, no `AGENTS.md`, and no `.cursor/rules/` folder. The two conditions therefore differ only in convention encoding, which is the variable the framework is designed to measure. The exact dependency surface and commit hashes used to build each condition are recorded in §6.2 for replication; the exact `create-next-app` CLI invocation and the post-creation edits for Condition B are kept in the baseline repository’s git history and are not duplicated in the bound thesis.

Table 6.2: Condition A (SmartKit at full setup) vs. Condition B (convention-naive scaffold) used in the pilot evaluation.

Dimension	Condition A (treatment)	Condition B (control)
Stack surface	SmartKit (Next.js 16, Drizzle, Biome, Zod, Resend, Better Auth)	create-next-app scaffold + full SmartKit dependency surface (Next.js 16, TypeScript, Tailwind CSS, Biome, Drizzle ORM, Zod 4.4.3, Resend, Better Auth); no src/features/ layout, no AGENTS.md, no .cursor/rules/
Convention envelope	FSD src/features/<name>/, AGENTS.md, server-only, colocated Zod in types/, Server Actions in actions/	No AGENTS.md, no src/features/, flat lib/ at root
Folder layout	Feature-scoped: components/, hooks/, actions/, lib/, types/, emails/ per feature	app/ (pages), lib/ (flat), components/ (flat)
AI tool context	Sees existing features/auth/, features/billing/ patterns; conventions live in repo files, not in separate docs	Sees only create-next-app default plus env placeholders; no SmartKit conventions to read
Pilot feature	Generate isEnabled(flagKey, userId?) utility + Zod schema + centralized test at src/tests/features/flags/	Generate same isEnabled(flagKey, userId?) utility; no SmartKit conventions to read

Condition A (SmartKit). Condition A uses the SmartKit codebase, which includes:

- **AGENTS.md** at the repository root: architectural conventions
- **.cursor/rules/** directory: cursor-specific instruction files
- **src/features/{name}/** structure: feature-based organization
- Existing example features (features/auth/, features/billing/) the AI sees before generating code
- Zod schemas colocated in features/{name}/types/
- Server Actions in features/{name}/actions/
- server-only imports as explicit server-boundary markers

Condition B (convention-naive scaffold). Condition B uses a baseline Next.js codebase produced by create-next-app with the same dependency surface as SmartKit (Next.js 16.2.12, TypeScript 5.7.3, Tailwind CSS 4.3.3, Drizzle ORM 0.45.2 with PostgreSQL, Zod 4.4.3, Biome 2.5.6, Resend 6.18.1, and better-auth 1.6.x) but no src/features/ layout,

no `AGENTS.md`, and no `.cursor/rules/`. TypeScript strict mode is enabled at parity with Condition A. The baseline represents a typical Next.js project without explicit architectural guidance.

The control comparison addresses the primary threat to internal validity: that observed scores on SmartKit reflect technology stack quality rather than convention adherence. By using the same dependency surface in both conditions, the experiment isolates the convention variable. In the executed pilot, the Cond B Type & Contract dimension showed a wider inter-run range (3 points across $n = 3$) than Cond A; this residual within-condition variance—not convention encoding—is the most plausible alternative explanation for the size of the score gap, and is one reason the DP verdicts in Section 6.2 are reported as descriptive rather than inferential. Cross-AI-tool replication, where version drift can be measured independently across tools, is identified as future work (Section 7.4).

6.2.2 Standardized Prompt

The executed pilot applied the standardized prompt (Listing 6.1) verbatim to both codebases.

```

1  Add a feature flag check utility for this Next.js codebase. The
2  utility should expose a single function 'isEnabled(flagKey, userId?)'
3  that returns a boolean.
4
5  Resolution order (highest priority first):
6    1. Per-user DB override, only if 'userId' is passed.
7    2. Per-flag environment variable 'FLAG_<KEY>=true|false'.
8    3. Default value (passed as the second argument).
9
10 Constraints:
11   - Cache the result in-memory for 5 minutes, keyed by flag and
12     (if present) userId. Cache must be process-local.
13   - The function should be a server-side utility; do not export it
14     to the browser.
15   - Define a 'FlagKey' type that the function accepts. Use a Zod
16     schema to derive it.
17   - Include at least 3 unit tests covering: env override, default
18     value, and cache hit.
19   - Use TypeScript strict mode.
20
21 You may add files anywhere in the repository that you think is
22 appropriate. The only requirement is that the public API is reachable
23 as 'isEnabled' and the type 'FlagKey' is exported.
```

Listing 6.1. Standardized prompt for the control comparison experiment, applied verbatim to both conditions.

The prompt intentionally avoids referencing SmartKit-specific conventions (*do not* say “for SmartKit”, “follow the feature-based layout”, “use `.server` naming”, or “refer to `AGENTS.md`”). The same prompt applies to both SmartKit and the convention-naïve scaffold, so any difference in output quality is attributable to the convention encoding rather than the prompt content. The full pre-registration record, including the frozen prompt hash, is reproduced in §6.2.

The same per-run pipeline was applied to all eight runs, with per-condition aggregates reported in Table 6.8:

1. Initialize a fresh copy of the codebase with the pre-registered tooling verified against a run-day checklist.
2. Invoke Cursor AI with the standardized prompt; start the time-to-correct clock.
3. Collect generated files, Biome violations, `tsc--noEmit` output, and artifact enumeration; snapshot the run in `runs/cond-<a,b>/run-<NN>/`.
4. Score each run on the three-dimension rubric; record per-run JSON plus manual cells; commit before the next run.

6.2.3 Run Schedule and Verdict Rule

The pilot used an adaptive 8-run schedule: three Cond B runs first, then Cond A, with the binary verdict rule governing whether Cond A extended to $N = 5$. The full schedule with tool snapshots and verdict counts is in Table 6.8; the per-minute timestamps live in the pilot audit trail released in the thesis GitHub repository (see `REPLICATION.md`). The verdict rule in force applies to the post-extension data: 5/5 confirmed; 0/5 refuted; 1–4/5 inconclusive. The original adaptive $N = 3$ rule was superseded by amendment 2026-09-01 (§6.5.2).

The pilot followed a pre-registered adaptive schedule, detailed in §6.2: the first three runs were executed in Condition B, and the inter-run range on Type & Contract Safety (0–3) exceeded the pre-registered continuation threshold, triggering Runs 4–5 in Condition A. Condition A was then extended from $n = 2$ to $n = 5$ (Runs 6–8) so the pre-registered binary verdict rule ($N = 5$ per condition) could issue a determinate outcome; the extension and its justification are logged in the amendment history (§6.2; full chronology in the pilot audit trail released in the thesis GitHub repository). The final tally is eight runs: $n = 5$ Condition A and $n = 3$ Condition B.

For each run treated as a binary outcome (pass = score $\geq 3/5$; fail = score $< 3/5$), the Wilson 95% confidence interval on the proportion of passing runs is computed by:

$$\frac{k + \frac{z^2}{2}}{n + z^2} \pm \frac{z}{n + z^2} \sqrt{\frac{k(n - k)}{n} + \frac{z^2}{4}} \quad (6.1)$$

where k is the number of passing runs, $n = 5$ (or $n = 3$ for Cond B), and $z = 1.96$ for 95% confidence. The Wilson interval is preferred over the normal approximation (Wald interval) for small n because it never produces bounds outside $[0, 1]$ and has better coverage properties when the true proportion is near 0 or 1. Results are reported as median [min, max] with Wilson 95% CI for the pass rate.

A second rater was scheduled to independently score 2 runs (1 per condition) using the same rubric, with Cohen’s $\kappa \geq 0.7$ as the threshold for rubric revision. The IRR pass was deferred within the Bachelor’s thesis timeline for lack of a second rater; manual cells in `runs/cond-<a,b>/run-<NN>/score.json` remain primary-rater only.

6.2.4 Scoring Rubric

Each run scores on three dimensions, each on a 0–5 integer rubric. The aggregate per-run score is the sum of three dimensions (0–15). Per-condition analysis reports median plus range, with Wilson 95% confidence intervals on the binary pass rate ($\geq 3/5$).

Dimension 1: Structural Integrity (0–5)

Level	Label	Description (all must hold)
0	Missing	No <code>isEnabled</code> function produced.
1	Severe	Function exists but in <code>app/</code> , <code>root lib/</code> , or <code>root /</code> . No <code>features/</code> folder used.
2	Partial	Function in <code>features/</code> subtree but in a wrong subfolder (e.g. <code>features/flags/index.ts</code> for the implementation, or <code>features/flags/components/</code>).
3	Acceptable	Function in <code>features/flags/lib/is-enabled.ts</code> (or colocated in another <code>features/<name>/lib/</code>) AND exports a named <code>isEnabled</code> function (default export also acceptable). File presence alone is insufficient; the export must exist.
4	Good	Level 3 + <code>server-only</code> import present in <code>is-enabled.ts</code> .
5	Excellent	Level 4 + public API barrel <code>features/flags/index.ts</code> re-exports <code>isEnabled</code> .

Table 6.3: Structural Integrity scoring rubric (0–5).

Dimension 2: Type & Contract Safety (0–5)

Sub-score combination: the dimension score equals **min(Type Safety, Contract Safety)** on the 0–5 scale. The weakest-link aggregation is asserted at design stage.

Type Safety sub-score

Level	Label	Description
0	Failing	<code>tsc --noEmit</code> produces > 5 errors.
1	Poor	3–5 <code>tsc</code> errors, or ≥ 3 <code>:any / asany</code> occurrences.
2	Fair	1–2 <code>tsc</code> errors, or 1–2 <code>any</code> occurrences.
3	Good	0 <code>tsc</code> errors, 0 <code>any</code> in <code>isEnabled / FlagKey</code> API, but <code>//@ts-ignore</code> or <code>branded-type-as-string</code> present.
4	Very good	0 <code>tsc</code> errors, 0 <code>any</code> in public API, 0 <code>@ts-ignore</code> . <code>FlagKey</code> is a string literal union.
5	Excellent	Level 4 + <code>FlagKey</code> is a Zod-derived enum (<code>z.enum([...])</code> or branded type with <code>z.brand<...>()</code>).

Table 6.4: Type Safety sub-score rubric (0–5).

Contract Safety sub-score

Level	Label	Description
0	None	No runtime validation: invalid <code>flagKey</code> reaches the cache layer.
1	Trivial	Validation exists but only as <code>if(typeof!=='string')throw</code> .
2	Manual	Manual union-type guard at function entry.
3	Zod present	Zod schema present in <code>features/flags/types/flags.ts</code> and parsed at function entry.
4	Typed result	Level 3 + parse error surfaces a typed result (not <code>throw</code>).
5	Single source	Level 4 + schema is the single source of truth for <code>FlagKey</code> (no parallel string literal union).

Table 6.5: Contract Safety sub-score rubric (0–5).

Dimension 3: Coverage & Completeness (0–5)

Level	Label	Description (all must hold)
0	No test	No <code>is-enabled.test.ts</code> file.
1	Minimal	Test file present but < 3 cases, or 3 cases but all <code>expect(true).toBe(true)</code> style.
2	Incomplete	≥ 3 test cases but no cache hit case.
3	Adequate	≥ 3 cases including env override + default value + cache hit. Cache implementation present (5-minute TTL, but the timing logic may be approximate).
4	Per-user covered	Level 3 + per-user override path is implemented (not stubbed — i.e. returns a value derived from the database, not a hard-coded <code>undefined</code> — and has a test.
5	Excellent	Level 4 + no <code>console.log</code> left in production path + test file at centralized path <code>src/tests/features/flags/is-enabled.test.ts</code> (not in <code>__tests__</code>).

Table 6.6: Coverage & Completeness scoring rubric (0–5).

6.2.5 Exclusion Criteria

ID	Exclusion Criterion
E1	AI assistant crashes, times out, or refuses to generate code
E2	Generated code contains syntax errors that prevent TypeScript compilation
E3	AI assistant generates files in programming languages other than TypeScript
E4	Run is interrupted by operator error (network failure, manual termination)
E5	AI assistant explicitly states it cannot complete the task due to missing dependencies or configuration

Table 6.7: Exclusion criteria for pilot runs.

All 8 executed runs completed without triggering E1–E5; the criteria are documented for replication rather than retrospective use.

6.3 Empirical Results

This section reports the pilot evaluation of the `isEnabled(flagKey, userId?)` utility under the pre-registered adaptive protocol, comparing Condition A (SmartKit with full convention encoding) against Condition B (convention-naïve scaffold). The final tally is 8 runs: $n = 5$ Condition A and $n = 3$ Condition B (Section 6.2.1). The pilot is reported as a worked example, not a hypothesis test: Two of the three directional predictions are confirmed outright (DP1, DP2); DP3 is confirmed on its Condition A clause and refuted on its Condition B clause under the pre-registered binary rule (5/5 Condition A runs in the predicted direction), but the per-arm sample remains far below the $n \approx 30$ needed for inferential validation, so cross-tool replication, larger- N validation, and developer-participant studies remain future work.

6.3.1 Per-run Scores

Task. The pilot applied the framework to a representative task: generating the `isEnabled(flagKey, userId?)` utility using Cursor AI. This utility resolves a feature flag with three override layers (per-user DB override, per-flag environment variable, default value) and caches the result in-memory for five minutes. As established in Section 6.1.1, the feature was chosen because it exercises none of SmartKit’s integrated services, eliminating integration-pattern confounding. The task instruction mirrored a realistic developer request without over-specifying implementation details, so as to evaluate the AI assistant’s ability to discover and apply existing conventions rather than merely follow explicit instructions.

Scoring. *Structural Integrity* was scored by running the `score-flag.ts` script against the generated files, counting violations per category (file placement, naming, Server/Client boundary). *Type & Contract Safety* was scored by running `tsc --noEmit` on the generated code and counting type errors and *any*-type occurrences. *Coverage & Completeness* was scored by enumerating generated files against a reference checklist (utility function, Zod

schema, centralized test at `src/tests/features/flags/`, public API barrel) and computing a coverage ratio. Each dimension was reported on the 0–5 rubric described in §6.1 (with the pre-registered rubric detailed in §6.2.4).

Pilot output and adaptive schedule. The pilot produced the frozen task prompt, generated-code snapshots for all 8 runs, a per-run scoring table, per-condition medians and Wilson 95% confidence intervals, and the full run log with the adaptive-schedule decision; all artefacts are released alongside this thesis and summarised in §6.2. The adaptive rule itself worked as follows: three runs were executed in Condition B first; because the inter-run range on Type & Contract Safety (0–3) exceeded the pre-registered continuation threshold, the protocol proceeded to Runs 4–5 in Condition A. The binary verdict rule was still indeterminate at $n = 2$ Condition A (it requires $N = 5$ per condition to issue *confirmed/refuted*), so Condition A was extended to $N = 5$ via Runs 6–8. This extension, and its date and justification, are recorded in the pilot audit trail (amendment log) released in the thesis GitHub repository.

Per-run scores. Manual ratings per the rubric in §6.2 (§6.2.4):

6.3.2 Per-condition Aggregate

Per-condition aggregate. Condition A medians [min, max]: Structural 5 [4, 5]; Type & Contract 4 [4, 5]; Coverage 3 [3, 3]. Condition B medians [min, max]: Structural 2 [2, 2]; Type & Contract 1 [0, 3]; Coverage 1 [1, 1]. The Condition B Type & Contract range (3) remains the largest variance observed; the other two dimensions were stable within each condition. Type & Contract scores widened slightly (from 0 range across Runs 4–5 to 1 across Runs 4–8) once Runs 6–8 entered the sample, but the median remained anchored at 4 and no run fell below the $\geq 3/5$ rubric threshold. These small per-arm samples are far below the $n \approx 30$ required for any frequentist inferential claim (see §7.4 for the sample-size discussion that bounds these verdicts). The Coverage cap at 3 for Condition A is a rubric-design consequence; the structural contradiction is analysed in §6.6.3.

6.3.3 Automated vs. Manual Cross-Check

Automated vs. manual score cross-check. For each run, the `score.json` artefact at `score.json` records both the `automated_scores` block (from `score-flag.ts`) and the `manual` block (primary rater). Across the eight runs, the two layers agree within ± 1 on every dimension except Coverage for Cond A runs 4–8 (post-correction Man Cov = 3 vs. Auto Cov = 5, $\Delta = 2$; see the “Note on cross-check methodology” below for why Man Cov was corrected from the first-pass 4–5 cells). They were cross-checked post-hoc rather than via blind inter-rater reliability (IRR; see Known Limitations in §7.2). When the two layers diverged on a given dimension, the discrepancy was logged in the run’s `notes` field and the run retained both values. Note: the `tooling_snapshot` field within `score.json` records the tools captured by the run-execution harness at session-open time; Run 4’s snapshot returned “unknown” for tool and model because the harness for Runs 1–5 was not yet configured to capture Cursor AI session metadata (this was added for Runs 6–8). Table 6.8 records the actual tool used per run, reconstructed from the audit log; this discrepancy is cosmetic and does not affect scoring. Table 6.9 below makes the cross-check auditable: every per-run pair is shown side-by-side so the cross-check is verifiable from the raw `score.json` files without re-running the harness (the Auto/Man cells shown are the post-correction cells for Cov runs 4–8; the pre-correction Man Cov values are preserved in `score.json`).

Table 6.8: Per-run pilot scores (Str / T&C / Cov, 0–5 rubric) across 8 runs: Condition A ($N = 5$, Runs 4–8) vs. Condition B ($N = 3$, Runs 1–3). Tool column shows the Cursor AI mode; per-minute timestamps and time-to-correct live in the pilot audit trail released in the thesis GitHub repository (REPLICATION.md). The Verdict column is the directional-prediction outcome under the binary rule (§6.2.3).

Run	Cond	Tool	Str	T&C	Cov	Verdict (DP1/2/3)	Notes
1	B	auto-mode	2	1	1	n/a / n/a / n/a	root lib/feature-flags.ts; no Zod
2	B	auto-mode	2	3	1	n/a / n/a / n/a	root lib/flags.ts; type alias only
3	B	auto-mode	2	0	1	n/a / n/a / n/a	root lib/feature-flag.ts; range Type=3 → PROCEED
4	A	auto-mode	5	5	3	incon. / incon. / incon.	features/flags/lib + Zod + server-only + 4 tests; per-user override stubbed (see †)
5	A	auto-mode	4	5	3	incon. / incon. / incon.	features/flags/lib + Zod + server-only + 3 tests; per-user override path dropped (see †)
6	A	agent-mode	5	4	3	conf. / conf. / conf.	features/flags/lib + Zod + server-only + 4 tests; per-user override stubbed (see †)
7	A	agent-mode	5	4	3	conf. / conf. / conf.	features/flags/lib + Zod (async variant) + 4 tests; per-user override stubbed (see †)
8	A	agent-mode	5	4	3	conf. / conf. / conf.	features/flags/lib + Zod + dual-Map cache + 4 tests; per-user override stubbed (see †)

† The Coverage value for these runs is 3 rather than the rubric-implied 5 because the generated override helper returns a hard-coded `undefined` instead of a database value; see the *Note on Coverage cells* below. All Cond A rows use the feature-based path `features/flags/lib/` with a Zod-derived `FlagKey` and the `server-only` import; all Cond B rows omit these.

Note on tool substitution. Runs 6–8 used `cursor/agent-mode` rather than the `cursor/auto-mode` of Runs 1–5; see §7.2 and §6.2 for full disclosure (per-run timestamps and tool-snapshot substitutions are recorded in the pilot audit trail released in the thesis GitHub repository).

Note on Coverage cells. The Coverage values reported in this table (Cov = 3 for Cond A runs 4–8) reflect a post-data-collection re-evaluation against rubric A.5.3 level 4, which requires the per-user override path to be *implemented, not stubbed* (i.e. returning a value derived from a database rather than a hard-coded `undefined`). The generated Cond A code at Runs 04, 05, and 08 returns a hard-coded `undefined` from the override helper (file: `runs/cond-a/run-NN/features/flags/lib/is-enabled.ts` for $NN \in \{04, 05, 08\}$), which places each run at rubric level 3 (“Adequate”) rather than 4 or 5. The `per-run.score.json` artefacts released alongside this thesis preserve the primary rater’s original manual cells at Coverage = 5; the table’s Cov = 3 reflects the stricter rubric interpretation applied to the table narrative. The directional prediction DP3 (test file at the centralized path `src/tests/features/flags/`, matching rubric A.5.3 level 5) is unaffected: tests were generated at `src/tests/features/flags/` in 5/5 Cond A runs and at repo root in 3/3 Cond B runs, exactly as the pre-registration predicted.

Run	Cond	Auto Str	Auto T&C	Auto Cov	Man Str	Man T&C	Man Cov
1	B	2	0	0	2	1	1
2	B	2	3	0	2	3	1
3	B	2	0	0	2	0	1
4	A	5	4	5	5	5	3
5	A	5	4	5	4	5	3
6	A	5	4	5	5	4	3
7	A	5	4	5	5	4	3
8	A	5	4	5	5	4	3

Table 6.9: Per-run automated vs. manual score cross-check across all eight pilot runs; maximum per-dimension absolute delta = 2 across 24 (run, dimension) pairs, driven entirely by the post-correction Coverage cells (Runs 4–8; see the “Note on cross-check methodology” below).

Note on cross-check methodology. Str = Structural Integrity, T&C = Type & Contract Safety, Cov = Coverage & Completeness. Each cell is read directly from the corresponding runs/cond-<a, b>/run-<NN>/score.json file (automated_scores block for the auto column, manual block for the man column). The Man Cov column reflects the *post-correction* manual cells (Cov = 3 for all Cond A runs 4–8), consistent with Table 6.8; the pre-correction manual Coverage values (5, 4, 5, 5, 5 for runs 4–8) were first-pass ratings against a rubric interpretation that did not strictly enforce rubric A.5.3 level 4’s “not stubbed” requirement on the per-user override path, and the corrected cells reflect the stricter interpretation applied uniformly across both tables in this thesis. The pre-correction cells remain preserved in the per-run score.json artefacts alongside this thesis for audit traceability. The maximum per-dimension absolute delta across all 24 (run, dimension) pairs is 2 (Auto Cov = 5 vs. Man Cov = 3 for Cond A runs 4–8); the other 22 pairs agree within ± 1 . The largest remaining divergences within ± 1 are: Run 5 Type & Contract (automated 4 vs. manual 5, a one-point judgment-call split) and Run 1/3 Type & Contract (automated 0 vs. manual 0–1, where manual cells credit a thin Zod-adjacent type alias the automated layer does not detect). Both layers were filled by the same primary rater (Runs 1–5) or by the AI agent acting as primary-rater proxy (Runs 6–8, transparently disclosed); no blind IRR pass was executed (see §7.2).

6.3.4 Directional Prediction Verdicts

DP verdicts. The binary verdict rule requires $k/5$ runs in the predicted direction for *confirmed*, $0/5$ for *refuted*, and reports $1\text{--}4/5$ as *inconclusive*.

- **DP1 (Structural placement in features/flags/lib).** Condition A: 5/5 in the predicted location; Condition B: 3/3 at root lib/ (i.e. 0/3 in Condition A’s predicted location). Wilson 95% CI: Condition A [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$); Condition B [0.44, 1.00] (half-width ≈ 0.28 , imprecise at $N = 3$). **Confirmed** (5/5) under the binary rule but not a precise inferential estimate.
- **DP2 (Zod-derived FlagKey enum).** Condition A: 5/5 used a Zod-derived FlagKey enum; Condition B: 3/3 used a raw string type alias. Wilson 95% CI: Condition A [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$); Condition B [0.44, 1.00] (half-width ≈ 0.28 , imprecise at $N = 3$). **Confirmed as instruction-following asymmetry** (5/5 Cond A complied with the directive, 3/3 Cond B did not) under the binary rule, but **not a precise inferential estimate** and **not a confirmation of convention-encoding benefit** on this dimension. The standardized prompt (Listing 6.1) instructs

both conditions to “use a Zod schema to derive” the `FlagKey` type, so the observed difference measures Condition B’s instruction-following failure (ignoring an explicit directive) rather than Condition A’s convention-encoding benefit; the DP2 result does not isolate the structural contribution from the prompting contribution, because both the convention (Zod) and the prompt directive (Zod) point in the same direction. See §7.2 for the full discussion of this bundled-effect limitation, and §7.4 for the design variation that would isolate the structural contribution (drop the shared Zod directive from the Cond B prompt).

- **DP3 (Centralized test path).** This prediction has two clauses: Condition A places the test at `src/tests/features/flags/` (predicted); Condition B produces no test file (predicted). Condition A: 5/5 colocated as predicted — **confirmed**. Wilson 95% CI: [0.57, 1.00] (half-width ≈ 0.21 , imprecise at $N = 5$). Condition B: 3/3 runs *did* produce a test file (at the repository root), the opposite of the prediction — **refuted**. Wilson 95% CI: [0.44, 1.00] for the absence of a test file (half-width ≈ 0.28 , imprecise at $N = 3$). This asymmetric outcome is reported honestly rather than retrofitted to a single verdict: the pre-registration did not anticipate that Condition B would consistently generate tests, and that is itself a finding worth noting.

One prediction is unambiguously confirmed (DP1, structural placement); one is confirmed as an instruction-following asymmetry, not as a convention-encoding benefit (DP2); and one is split, confirmed on its Condition A clause and refuted on its Condition B clause (DP3). Runs 6–8, generated under a substituted tool snapshot (Table 6.8), reproduce the same pattern as Runs 4–5, which is why they are retained in the aggregate; the substitution does not change the rubric or the scoring procedure, only the assistant that produced the code. The verdicts above should be read as a single worked pilot on one feature and one tool family, not as a validated cross-tool result — the limitations of this design are discussed in Section 7.2.

Wilson-CI precision caveat. The Wilson 95% intervals above are computed from small per-arm samples ($N = 5$ for Condition A, $N = 3$ for Condition B); the half-width of a Wilson interval at these sample sizes is approximately 0.21 for Condition A and 0.28 for Condition B, so a single-condition CI cannot reliably distinguish proportions below 0.5 from those at 0.8 or higher, and the verdicts should not be over-interpreted as inferential claims. The precision floor for the executed pilot and the sample-size considerations that follow from it are discussed in §7.4.

6.4 Discussion of Trade-offs

This section reads the DP verdicts against the rubric mechanics rather than against the pre-registration narrative. The three subsections below name one mechanism that the framework captures, three mechanisms that the framework fails to isolate cleanly, and the statistical envelope that bounds how much any of those mechanisms can be claimed from the executed sample. Threats-to-validity items (IRR deferral, unblinded scoring, tool substitution, asymmetric N , rubric-design contradiction) are catalogued in §6.6 and not re-litigated here.

6.4.1 Where the framework wins: DP1 as a measurement success

DP1’s 5/5 Condition A outcome on structural placement is a measurement success of the rubric, not a causal claim about the underlying convention. The mechanism the framework captures is the discovery path: when an assistant is given a Next.js codebase with

a ‘features/flags/lib/’ convention already established by sibling features (e.g. the existing ‘is-enabled.ts’ placement documented in §6.1.1), it reads those siblings, recognises the convention, and replicates it at the predicted location. Condition B’s 3/3 placement at root ‘lib/’ is the mirror image: in the absence of a sibling convention to discover, the assistant defaults to a generic location. The Wilson 95% CI on Condition A is [0.57, 1.00] (half-width ≈ 0.21) and on Condition B is [0.44, 1.00] (half-width ≈ 0.28); the gap between the two arms is what the framework measures, and the gap is consistent with the rubric’s design intent: Structural Integrity rewards convention-conformity, and the two conditions differ by design in the density of discoverable conventions.

The mechanism is observational at this point: the framework records that the convention was followed, not why the assistant chose to follow it. A future controlled design (dropping all explicit prompts that hint at ‘features/flags/’, comparing assistant behaviour against a sibling-less SmartKit branch) would be required to attribute the conformity to convention-reading rather than to instruction-following. Such a design is sketched in §7.4 as a follow-up variation; the pilot as executed does not support that attribution.

6.4.2 Where the framework loses

Three patterns appear in the DP verdicts that the rubric, as specified, does not cleanly isolate. Each is reported below as a measurement gap, not as a defect in the rubric’s purpose.

DP2 is an instruction-following asymmetry, not a convention-encoding benefit. The standardized prompt (Listing 6.1) instructs both conditions to “use a Zod schema to derive” the FlagKey type. Condition A complied with the directive in 5/5 runs; Condition B did not comply in 3/3 runs (raw ‘string’ alias instead). The rubric scores this dimension identically in both conditions (a Zod-derived type scores 5, a raw alias scores 0–1), so the dimensional verdict (Condition A wins) is correct. What the verdict does not separate is the structural contribution from the prompting contribution: both the codebase convention (Zod is the existing type-source pattern in SmartKit) and the shared prompt directive (“use a Zod schema”) point in the same direction, so a compliant Condition A run cannot be attributed to either factor in isolation. The Cond B non-compliance is informative because it shows the directive was available and Condition B did not follow it, but that observation alone does not tell a reader whether Condition A’s compliance was caused by the convention, the directive, or both. The bundled-effect limitation is acknowledged in §7.2 and the design variation that would isolate the structural contribution (drop the shared Zod directive from the Cond B prompt) is recorded in §7.4.

DP3-B is refuted: Condition B generated tests when the pre-registration predicted it would not. Condition B’s 3/3 root-level test-file generation refutes the second clause of DP3. The pre-registration predicted Condition B would produce no test file; all three runs produced one. The mechanism the rubric fails to capture here is general-purpose test-generation behaviour: an assistant given a TypeScript utility and an instruction to “write tests if appropriate” will, under most prompting configurations, generate some test file rather than none. The original DP3 design assumed Condition B would treat the prompt as optional with respect to tests; that assumption was incorrect for the prompt phrasing used. Condition A’s 5/5 prediction match on placement (centralized at ‘src/tests/features/flags/’) is unaffected, and that placement is what DP3 was designed to measure; the refuted clause is reported honestly rather than collapsed into a single “confirmed” verdict.

Coverage in Condition A is capped at 3. This is a rubric-design consequence (level 4–5 of rubric A.5.3 require a non-stubbed per-user override path, which the pilot feature structurally cannot produce by design). The structural contradiction is analysed in §6.6.3.

6.4.3 Statistical context: $n = 5$ and $n = 3$ are not inferential

The Wilson 95% half-widths reported in §6.3.4 are reproduced here verbatim to anchor the Discussion in the executed numbers, not in summary prose: Condition A half-width ≈ 0.21 (CI [0.57, 1.00] on 5/5); Condition B half-width ≈ 0.28 (CI [0.44, 1.00] on 3/3). At these widths, a single-condition Wilson interval cannot distinguish a true proportion of 0.50 from one of 0.80; the DP verdicts are therefore binary-rule outcomes (a 5/5 count resolves “confirmed” under the rule’s $N = 5$ requirement) rather than parameter estimates with confidence. The DP verdicts in §6.3.4 should be read as feasibility-stage evidence that the framework can measure the predicted contrast under the executed conditions, not as estimates of that contrast in a population of AI assistants or codebases.

A reader asking “how big is the Condition A effect?” is asking a question the executed pilot does not answer. The conditions under which the effect size would become estimable (per-arm $N \approx 30$, symmetric N across both arms, two or more tool families) are recorded as future work in §7.4 and as a Known Limitation in §7.2. The Discussion above reads the DP verdicts against the rubric mechanics for what they show at the executed sample size; it does not extrapolate.

6.5 Pre-registration and Amendments

6.5.1 Lock, Amendment Procedure, and Snapshot Limitation

Lock. The pre-registration document was locked on 2026-09-02T05:23:00Z before any pilot runs were executed. Five commitments are recorded:

- The prompt template (§4) will not be modified
- The scoring rubric (§6.2.4) will not be modified except under inter-rater reliability failure
- The exclusion criteria (§6.2.5) will not be modified
- The run schedule (§6.2.3) will not be modified except for exclusions
- The adaptive $N = 3$ verdict rule (§6.2.3) will not be modified

Three amendments are permitted by the lock: rubric clarification triggered by IRR failure, exclusion-triggered schedule replacement, and typographical corrections that do not change scientific meaning. The SHA-256 hash of the locked document is recorded in §6.5.1.

Snapshot limitation. This pre-registration documents the pilot design as executed, not a prospective commitment formed before any preliminary work. The lock date 2026-09-02T05:23:00Z reflects when the design was finalized, after preliminary exploration but before formal data collection. The lock prevents rubric scoring and analysis decisions from being influenced by observed results; it does not represent a traditional prospective pre-registration timeline.

6.5.2 Pilot Run Log and Amendment Record

Pilot run log. The pilot used an adaptive schedule: three Cond B runs to establish a baseline Type-range signal, then Cond A. Runs 1–5 used Prompt B on cursor/auto-mode; Runs 6–8 used the same prompt on cursor/agent-mode (amendment 2026-09-06). The switch to Agent mode was forced by tool availability, not deliberate experimental design. The per-run log with timestamps and totals lives in the pilot audit trail released in the thesis GitHub repository; §6.3 reports the consolidated per-run scores directly.

Amendment log. The pre-registration was amended ten times between 2026-09-01 and 2026-09-08. Three changes matter most for reading Chapter 6: (i) the Cond A arm was extended from $n = 2$ to $n = 5$ so the binary verdict rule could resolve (amendment 2026-09-06); (ii) the rubric was widened from 3-level (0–2) to 6-level (0–5) per dimension to match the `score-flag.ts` implementation (amendment 2026-09-01); (iii) the IRR pass was deferred for lack of a second rater within the Bachelor’s thesis timeline (amendment 2026-09-04, reflected in §7.2). The full chronology is in the same audit trail release (amendment log).

Content hashes. SHA-256 hashes for the locked pre-registration, the post-amendment content, and the Prompt B template are recorded in the pilot audit trail released in the thesis GitHub repository (`REPLICATION.md`) for replication; they are not reproduced in the body of this chapter.

6.6 Threats to Validity

The threats-to-validity taxonomy below organizes the limitations of the pilot under six categories drawn from empirical software engineering (External Validity, Construct Validity, Internal Validity, Temporal Validity, Conclusion Validity, and Instrumentation). The result-bound summary of known limitations and protocol deviations is in §7.2; this section is the analytical taxonomy, with each category discussed against the executed pilot rather than the pre-registered design.

6.6.1 External Validity (Generalizability)

External Validity / Generalizability. The SmartKit implementation focuses exclusively on Next.js App Router. Architectural patterns specific to Next.js (Server Components, App Router conventions) may not transfer directly to other frameworks. The cross-framework transfer analysis (§7.4) provides a conceptual roadmap for Nuxt and Remix instantiation. Future work should evaluate generalizability across Nuxt and Remix by applying the same three-dimension framework with framework-specific checklists; the cross-section is at present unverified.

Lab-to-Production Gap. The pilot was executed in a single-author, single-tool, single-feature setting; whether the framework’s directional predictions transfer to multi-developer production codebases with team-level code review and longer feedback loops remains unquantified. The deployed artefact, however, is publicly released, which permits replication by independent groups; this scope limitation thus delimits the precision of the present claims rather than blocking external replication.

Scope limitations. Coverage of the framework’s three dimensions was demonstrated on one representative feature (`isEnabled(flagKey, userId?)`); no claim is made about

how the rubric discriminates across heterogeneous feature surfaces (e.g. authentication flows, billing integrations, real-time collaboration). The asymmetric- N disclosure in §6.3 further bounds how the cross-condition comparison should be read; the boundary is the pilot feature, not Next.js boilerplate in general.

6.6.2 Construct Validity

Construct Validity. The three evaluation dimensions are derived from design principles that address the gap between output-oriented benchmarks and framework-specific quality dimensions. The foundation comes from a structured literature search (§3), but the specific dimension selection and rubric cutoffs are design choices rather than empirically validated constructs. The rubric cutoffs (0–1: serious violations, 1–2: minor, 2–3: acceptable, 3–4: good, 4–5: excellent) are asserted mappings to developer outcomes and have not been calibrated against an inter-rater reliability study. The pilot protocol (§6.2) documents the scoring procedure for future empirical execution; multi-run evaluation, IRR with $\kappa \geq 0.7$, and criterion-referenced validation remain follow-up steps. This categorization as Construct Validity delimits the precision of the rubric to its design-stage intent rather than its validated construct status.

6.6.3 Internal Validity

Rubric design flaw: Coverage levels 4–5 are unreachable for the pilot feature by construction. The Coverage & Completeness rubric (§6.2.4, Table 6.6) defines level 4 as requiring the per-user override path to be “implemented (not stubbed — i.e. returns a value derived from the database)” and level 5 as additionally requiring the centralized test path. The pilot feature (`isEnabled(flagKey, userId?)`) was chosen specifically to avoid any SmartKit integration including the database layer (§6.1, §6.1.1), so the per-user DB override path is structurally out of scope for any feature meeting the no-integration selection criterion. The result is a *structural contradiction* between the rubric and the confound-control design: rubric levels 4–5 are unreachable for the very feature the pilot chose to evaluate *by construction*, not as a contingent scoring error. The executed pilot therefore reports Coverage scores capped at 3 for all Cond A runs (Table 6.8); these $\text{Cov} = 3$ values reflect the design contradiction itself, not a finding about AI capability. Fixing this confound requires either (a) revising the rubric to remove the DB-implementation requirement from levels 4–5, or (b) selecting a pilot feature that integrates at least one SmartKit service and re-running the asymmetric- N analysis; both are documented as future work (§7.4).

Protocol Deviations (Internal-Validity Subset). Five protocol-affecting decisions bound how the within-condition results should be read; the full audit log with dates and justifications is in the pilot audit trail released in the thesis GitHub repository, and the result-bound summary is in §7.2. (i) *IRR deferral*: no second rater was available within the Bachelor’s thesis timeline, so all manual scores are single-rater. (ii) *Tool substitution*: Runs 6–8 used `cursor/agent-mode` rather than `cursor/auto-mode` (forced by tool availability, not experimental design). (iii) *N -extension asymmetry*: Condition A was extended from $n = 2$ to $n = 5$ post-data-collection so the binary verdict rule could issue a determinate outcome. This is the bias risk most likely to surface during defense; the amendment was logged but the bias is not eliminated by logging. (iv) *Verdict-rule shift*: the adaptive $N = 3$ rule was replaced mid-pilot by the binary $N = 5$ rule (§6.2.3); the lock permitted rubric and schedule amendments but not protocol-rule amendments, so this falls outside the categories

enumerated in §6.5.1. (v) *DP3-B refuted outright*: the Cond B clause of DP3 was falsified when all three Cond B runs produced root-level test files, opposite to the prediction.

Protocol-rule violation: N-extension and verdict-rule shift exceeded permitted amendments. The N-extension to Condition A ($n = 2 \rightarrow 5$) and the shift from the adaptive $N = 3$ verdict rule to the binary rule (§6.2.3) fall outside the amendments the pre-registration lock explicitly permits: the lock (§6.5.1) allows only IRR-triggered rubric revision and exclusion-triggered schedule replacement, not protocol-rule changes motivated by indeterminate verdicts. This is a protocol-rule violation of the same commitment device that otherwise constrains the pilot; the substantive effect is captured by the bias-risk disclosure above, and the formal effect is that the pre-registration’s amendment procedure (see pilot audit trail released in the thesis GitHub repository, amendment log) records these changes under the “permitted amendments” clause in form, while in substance they constitute a fourth category — protocol-rule amendment — that the lock did not enumerate. Future replication should add “protocol-rule amendment” as a fourth permitted-amendment category or, preferably, redesign the verdict rule so that an indeterminate result can be reported as inconclusive without requiring asymmetric N-extension.

Tool-dependent, one-sided confirmation. Three post-hoc decisions bound the DP verdicts. First, the adaptive extension to $N = 5$ was applied to Condition A only; Condition B remained at $N = 3$ because its own adaptive trigger (the Type & Contract range exceeding the continuation threshold) had already fired at $n = 3$, so the binary verdict rule could resolve Condition A but not Condition B. *The asymmetric N-extension (Cond A: $n = 5$ post-hoc; Cond B: $n = 3$) is a bias risk accepted for feasibility reasons within a Bachelor’s thesis timeline.* The pattern — collect data, observe the verdict rule cannot resolve for one arm, extend only that arm in the direction that produces “confirmed” — is similar to optional stopping / N -hacking; the reported confirmations describe the Condition A arm under the modified design, not a symmetric two-condition inferential result. Second, three of the five Condition A runs (Runs 6–8) were generated under a different AI-tool snapshot than Runs 1–5 (Table 6.8); excluding them would leave Condition A at $N = 2$, below the verdict rule’s threshold, so the *confirmed* status of DP1 and DP2 depends on treating Runs 6–8 as comparable to Runs 4–5. The scores from Runs 6–8 reproduce the Runs 4–5 pattern under an unchanged rubric and prompt, which is the basis for retaining them; this is a single worked pilot on one feature and one tool family, not a controlled cross-tool replication. Third, DP3’s Condition B clause (predicting no test file) was falsified: all three Condition B runs produced a test file, which the pre-registration did not anticipate. The full chronology, with dates and reasoning for each decision, is recorded in the pilot audit trail released in the thesis GitHub repository (amendment log).

Design confounds and deferred validation. Two further limitations bound how the pilot’s numbers should be read. The locked run schedule executed all three Condition B sessions before any Condition A session, so time-of-day, session-memory, and experimenter-familiarity confounds cannot be ruled out; because the Condition A block itself spans two tool snapshots, the run-order confound and the tool-snapshot difference are partially entangled for that condition, and a fully randomized run order is left as future work. Separately, every score in Table 6.8 reflects a single primary rater (see Known Limitations in §7.2); the scores remain reproducible from the underlying run artefacts, but have not been cross-checked against independent judgement.

6.6.4 Temporal Validity

Temporal Validity. The pilot was executed within a fixed Bachelor’s thesis timeline (2026-09-01 to 2026-09-08; the full chronology is in the pilot audit trail released in the thesis GitHub repository); whether the framework’s directional predictions remain stable as AI coding assistants evolve (Cursor auto-mode / agent-mode, underlying model versions, Next.js minor releases) is unquantified. The amendment log already records a tool-snapshot boundary between Runs 5 and Runs 6 within the same condition; this single pilot therefore delimits the precision of DP verdicts to the tool snapshot in force at execution time, and warrants future replication under current tool snapshots before the framework’s verdicts are treated as time-invariant.

6.6.5 Conclusion Validity

Alternative Solutions / Alternative Explanations. The pilot’s directional predictions are explained in terms of the three-dimension framework’s structural conventions; whether the same directional verdicts would emerge under alternative solution strategies (prompting-only, linter-based enforcement, enhanced documentation) is left as future work (§7.4). The confirmed DP1 and DP2 verdicts cannot, by themselves, distinguish “structural convention-encoding benefit” from “instruction-following asymmetry that the structural convention incidentally enables”, because Cond A’s prompt bundles both the AGENTS.md structural directives and the explicit instruction-following cues that Cond B’s prompt also contains. The bundled-effect limitation is disclosed here rather than retrofitted into a clean causal story; future work should re-run DP2 with the Cond B prompt removing the shared Zod directive to isolate the structural convention-encoding contribution from the prompting contribution.

6.6.6 Instrumentation

Scoring procedure feasibility vs. rubric-level validity. The pilot demonstrates *scoring procedure feasibility*. The framework’s `score-flag.ts` script (and the per-run JSON artefacts alongside this thesis) reproduces the same 0–5 rubric scores deterministically from the run snapshots, and a single primary rater’s manual scores agree with the automated layer within ± 2 across all eight runs (Coverage, post-correction; see §6.3 “Automated vs manual cross-check”). What the pilot does *not* demonstrate is *rubric-level validity*. The claim that rubric level 5 means “excellent boilerplate” and level 0 means “missing artifact” in any developer-relevant sense is asserted rather than calibrated. The cutoffs (0–1 serious violations, 1–2 minor, 2–3 acceptable, 3–4 good, 4–5 excellent) are asserted mappings to developer outcomes, not calibrated against an inter-rater reliability study, criterion-referenced comparison, or developer-experience measure; IRR with Cohen’s $\kappa \geq 0.7$ was deferred for lack of a second rater within the Bachelor’s thesis timeline. Future validation-stage replication ($n \geq 30$ per condition, two raters, criterion-referenced comparison against developer-reported quality) is required to establish rubric-level validity; until then, the rubric scores should be read as ordinal, reproducible measurements of the framework’s three dimensions, not as developer-relevant quality grades.

Known Limitations (Instrumentation Subset). Three identified limitations bound every quantitative claim in this thesis and are referenced from §7.2 rather than restated at each location. (a) *Small per-arm sample size*: $n = 5$ Condition A, $n = 3$ Condition B, with no frequentist inferential claim supported at these sample sizes (see §7.4 for the sample-size calculation that places validation-stage replication at $n \approx 30$ per condition). (b) *Deferred*

inter-rater reliability: the pre-registered IRR pass with Cohen’s $\kappa \geq 0.7$ was deferred for lack of a second rater within the Bachelor’s thesis timeline; all manual scores in Table 6.8 are single-rater, and the deferral is recorded in the pilot audit trail released in the thesis GitHub repository (§6.2). (c) *Unblinded primary-rater scoring*: the same author designed the rubric, built the `score-flag.ts` script, and produced SmartKit, creating a confirmation-bias risk that blinding would have prevented. The Coverage rubric’s post-hoc re-evaluation (Cov 4–5 → 3, see §6.3 “Per-condition aggregate”) is the most visible instance of a judgment-call that blinding would have governed.

Scoring was not blinded to condition. The primary rater was aware of which condition (A vs. B) each run belonged to at scoring time, and the same author designed the rubric, built the `score-flag.ts` script, and produced SmartKit. This limitation is item (c) in the Known Limitations block above; the elaboration below explains why blinding would have prevented the specific judgment-calls that arose. Even with the rubric cells written down in advance, a rater who knows the framework’s directional predictions and who also wrote the rubric can unconsciously map ambiguous cells in the predicted direction. This elaboration is particularly acute for the Coverage rubric’s post-hoc re-evaluation (§6.3, “Per-condition aggregate”), where first-pass scores of 4–5 for Cond A runs were reinterpreted downward to 3 after inspection of the actual generated code. The reinterpretation was applied in good faith and is documented transparently, but the change itself illustrates exactly the kind of judgment-call that blinding would have prevented. Mitigation for future work: each run’s files should be stripped of condition-identifying paths (e.g., `features/flags/` vs. `root lib/`) and re-labelled before scoring, with condition membership recovered only after the scores are committed to `score.json`. The current pilot’s per-run JSON artefacts are preserved so the re-scoring can be done ex post if a second rater becomes available.

7. Conclusion and Future Work

7.1 Summary

This thesis proposes a three-dimensional evaluation framework (Structural Integrity, Type & Contract Safety, Coverage & Completeness) for assessing AI-generated Next.js boilerplate, and instantiates it in SmartKit, an open-source Next.js App Router boilerplate with feature-based modularity, explicit Server/Client boundaries, and type-safe contracts. An adaptive pilot ($n = 5$ Cond A, $n = 3$ Cond B) on one representative feature produces feasibility-stage evidence that one of three directional predictions is confirmed outright (DP1, structural placement), one is confirmed as an instruction-following asymmetry rather than as a convention-encoding benefit (DP2), and one is asymmetric between its two clauses (DP3 confirmed Cond A, refuted Cond B); the full verdicts and disclosed protocol adaptations are in §6.2 and §7.2. Quantitative claims are bounded by the Known Limitations consolidated in §7.2 — small per-arm samples, deferred IRR, and unblinded scoring — which the thesis treats as the canonical reference rather than restating them at each reporting point. The framework, artefact, and executed pilot are intended as a foundation for larger- N replication, cross-tool validation, and developer-participant studies, all documented in §7.4.

7.2 Limitations and Threats to Validity

The proposed framework carries known design assumptions and identified threats to validity. The analytical taxonomy with the full discussion is in §6.6 (Threats to Validity); this section is the canonical result-bound summary, restricted to the limitations most likely to surface during defence and replication.

Rubric design flaw: Coverage levels 4–5 are unreachable for the pilot feature by construction. The structural contradiction — rubric A.5.3 levels 4–5 require a non-stubbed per-user override path, which the pilot feature structurally cannot produce by design — is analysed in §6.6.3. The executed pilot therefore reports $\text{Cov} = 3$ for all Cond A runs (Table 6.8), reflecting the design contradiction rather than a finding about AI capability.

Known Limitations (applies to all pilot results in §6 and §7). Three identified limitations bound every quantitative claim in this thesis and are referenced from §6.6 (Threats to Validity) and §7.2 rather than restated at each location. (a) *Small per-arm sample size*: $n = 5$ Condition A, $n = 3$ Condition B, with no frequentist inferential claim supported at these sample sizes (see §7.4 for the sample-size calculation that places validation-stage replication at $n \approx 30$ per condition). (b) *Deferred inter-rater reliability*: the pre-registered IRR pass with Cohen’s $\kappa \geq 0.7$ was deferred for lack of a second rater within the Bachelor’s thesis timeline; all manual scores in Table 6.8 are single-rater, and the deferral is recorded in the pilot audit trail released in the thesis GitHub repository. (c) *Unblinded primary-rater scoring*: the same author designed the rubric, built the `score-flag.ts` script, and produced SmartKit, creating a confirmation-bias risk that blinding would have prevented. The Coverage rubric’s post-hoc re-evaluation ($\text{Cov } 4\text{--}5 \rightarrow 3$, see §6.6.3) is the most visible instance of a judgment-call that blinding would have governed.

Protocol Deviations (Public Summary). Five protocol-affecting decisions made during the pilot are surfaced here for the reader who reaches this section without opening §6.2;

the full audit log with dates and justifications is in the pilot audit trail released in the thesis GitHub repository. (i) *IRR deferral*: no second rater was available within the Bachelor's thesis timeline, so all manual scores are single-rater. (ii) *Tool substitution*: Runs 6–8 used `cursor/agent-mode` rather than `cursor/auto-mode` (forced by tool availability, not experimental design). (iii) *N-extension asymmetry*: Condition A was extended from $n = 2$ to $n = 5$ post-data-collection so the binary verdict rule could issue a determinate outcome. This is the bias risk most likely to surface during defense; the amendment was logged but the bias is not eliminated by logging. (iv) *Verdict-rule shift*: the adaptive $N = 3$ rule was replaced mid-pilot by the binary $N = 5$ rule (§6.2.3); the lock permitted rubric and schedule amendments but not protocol-rule amendments, so this falls outside the categories enumerated in §6.5.1. (v) *DP3-B refuted outright*: the Cond B clause of DP3 was falsified when all three Cond B runs produced root-level test files, opposite to the prediction. Honesty requires acknowledging this falsification in narrative form even though the pre-registration did not anticipate it; the binary verdict rule reported it as refuted rather than retrofitting a single-clause outcome.

Construct Validity. The three evaluation dimensions are derived from design principles that address the gap between output-oriented benchmarks and framework-specific quality dimensions. The foundation comes from a structured literature search (Section 3), but the specific dimension selection and rubric cutoffs are design choices rather than empirically validated constructs. The rubric cutoffs (0–1: serious violations, 1–2: minor, 2–3: acceptable, 3–4: good, 4–5: excellent) are asserted mappings to developer outcomes and have not been calibrated against an inter-rater reliability study. Mitigation: the pilot protocol (Section 6.2) documents the scoring procedure for future empirical execution. Multi-run evaluation, IRR with $\kappa \geq 0.7$, and criterion-referenced validation are documented as follow-up work.

External Validity / Generalizability. The SmartKit implementation focuses exclusively on Next.js App Router. Architectural patterns specific to Next.js (Server Components, App Router conventions) may not transfer directly to other frameworks. The cross-framework transfer analysis (Section 7.4) provides a conceptual roadmap for Nuxt and Remix instantiation. Future work should evaluate generalizability across Nuxt and Remix by applying the same three-dimension framework with framework-specific checklists; the cross-section is at present unverified.

Alternative Solutions. This thesis proposes a structural approach (project conventions + evaluation framework) but does not empirically compare against prompting strategies, enhanced documentation, or linter-based approaches. The rationale for the structural approach is discussed in Section 3; the Related Work section explicitly notes that the Cond A advantage observed in the pilot is the *bundled* effect of structural conventions (feature-based folder layout, server-only boundaries, colocated Zod schemas, central test directory) operating *together with* prompting (the AI's exposure to the AGENTS.md instructions and to existing example features such as `features/auth/` and `features/billing/`) and tool enforcement (Biome lint configuration, Zod-driven TypeScript strict mode, Drizzle schema generation). The executed pilot does not isolate the "structural" component from these prompting and tooling layers; this empirical de-bundling is left as future work. Mitigation: the Related Work section provides design-stage reasoning only, and the comparative empirical study is documented as future work in Section 7.4.

Lab-to-Production Gap. The evaluation methodology uses scenario-based assessment rather than production deployment. Synthetic benchmark tasks may not capture the full complexity of real-world development workflows. *SmartKit's integrations are implemented but have not been audited for production deployment*: no security review, no load testing,

and no penetration testing were performed; the framework supports convention measurement and developer starting-point usability, but the SmartKit artifact should not be deployed in production environments without additional hardening. Mitigation: the framework addresses the lab-evaluation concern through trajectory-based evaluation (assessing code generation process, not just output), multiple runs to handle non-determinism, and planned developer participant studies following established methodology (Peng et al., 2023; Tariq et al., 2026).

Scope limitations. The evaluation focuses exclusively on Next.js, and findings may not generalize to other frameworks. Longitudinal effects of AI-assisted development remain unexplored.

Scoring procedure feasibility vs. rubric-level validity. The pilot demonstrates *scoring procedure feasibility* — the framework’s `score-flag.ts` script (and the per-run JSON artefacts alongside this thesis) reproduces the same 0–5 rubric scores deterministically from the run snapshots, and a single primary rater’s manual scores agree with the automated layer within ± 2 across all eight runs (Coverage, post-correction; see §6.3.3 (Automated vs manual cross-check)). What the pilot does *not* demonstrate is *rubric-level validity* — the claim that rubric level 5 means “excellent boilerplate” and level 0 means “missing artifact” in any developer-relevant sense. The cutoffs (0–1 serious violations, 1–2 minor, 2–3 acceptable, 3–4 good, 4–5 excellent) are asserted mappings to developer outcomes, not calibrated against an inter-rater reliability study, criterion-referenced comparison, or developer-experience measure; IRR with Cohen’s $\kappa \geq 0.7$ was deferred for lack of a second rater within the thesis timeline. Future validation-stage replication ($n \geq 30$ per condition, two raters, criterion-referenced comparison against developer-reported quality) is required to establish rubric-level validity; until then, the rubric scores should be read as ordinal, reproducible measurements of the framework’s three dimensions, not as developer-relevant quality grades.

Tool-dependent, one-sided confirmation. Three post-hoc decisions bound the DP verdicts. First, the adaptive extension to $N = 5$ was applied to Condition A only; Condition B remained at $N = 3$ because its own adaptive trigger (the Type & Contract range exceeding the continuation threshold) had already fired at $n = 3$, so the binary verdict rule — which requires $N = 5$ per condition to resolve — could resolve Condition A but not Condition B. *The asymmetric N-extension (Cond A: $n = 5$ post-hoc; Cond B: $n = 3$) is a bias risk accepted for feasibility reasons within a Bachelor’s thesis timeline.* The pattern — collect data, observe the verdict rule cannot resolve for one arm, extend only that arm in the direction that produces “confirmed” — is similar to optional stopping / N -hacking; the reported confirmations describe the Condition A arm under the modified design, not a symmetric two-condition inferential result. Second, three of the five Condition A runs (Runs 6–8) were generated under a different AI-tool snapshot than Runs 1–5 (Table 6.8); excluding them would leave Condition A at $N = 2$, below the verdict rule’s threshold, so the *confirmed* status of DP1 and DP2 depends on treating Runs 6–8 as comparable to Runs 4–5. The scores from Runs 6–8 reproduce the Runs 4–5 pattern under an unchanged rubric and prompt, which is the basis for retaining them; this is a single worked pilot on one feature and one tool family, not a controlled cross-tool replication. Third, DP3’s Condition B clause (predicting no test file) was falsified: all three Condition B runs produced a test file, which the pre-registration did not anticipate. The full chronology, with dates and reasoning for each decision, is recorded in the pilot audit trail released in the thesis GitHub repository (amendment log).

Protocol-rule violation: N-extension and verdict-rule shift exceeded permitted amendments. Additionally, the N-extension to Condition A ($n = 2 \rightarrow 5$) and the shift from the adaptive $N = 3$ verdict rule to the binary rule (§6.2.3) fall outside the amendments

the pre-registration lock explicitly permits: the lock (§6.5.1) allows only IRR-triggered rubric revision and exclusion-triggered schedule replacement, not protocol-rule changes motivated by indeterminate verdicts. This is a protocol-rule violation of the same commitment device that otherwise constrains the pilot, and it is disclosed here alongside the bias risk above rather than treated as if the lock procedure had authorized it. The substantive effect of the violation is captured by the bias-risk disclosure; the formal effect is that the pre-registration’s amendment procedure (see pilot audit trail released in the thesis GitHub repository, amendment log) records these changes under the “permitted amendments” clause in form, while in substance they are a fourth category — protocol-rule amendment — that the lock did not enumerate. Future replication should add “protocol-rule amendment” as a fourth permitted-amendment category or, preferably, redesign the verdict rule so that an indeterminate result can be reported as inconclusive without requiring asymmetric N-extension.

Design confounds and deferred validation. Two further limitations bound how the pilot’s numbers should be read. The locked run schedule executed all three Condition B sessions before any Condition A session, so time-of-day, session-memory, and experimenter-familiarity confounds cannot be ruled out, and — because the Condition A block itself spans two tool snapshots — the run-order confound and the tool-snapshot difference are partially entangled for that condition; a fully randomized run order is left as future work. Separately, every score in Table 6.8 reflects a single primary rater (see Known Limitations in §7.2); the scores remain reproducible from the underlying run artefacts, but have not been cross-checked against independent judgement.

Scoring was not blinded to condition. The primary rater was aware of which condition (A vs. B) each run belonged to at scoring time, and the same author designed the rubric, built the `score-flag.ts` script, and produced SmartKit. This limitation is item (c) in the Known Limitations block above; the elaboration below explains why blinding would have prevented the specific judgment-calls that arose. Even with the rubric cells written down in advance, a rater who knows the framework’s directional predictions and who also wrote the rubric can unconsciously map ambiguous cells in the predicted direction. This elaboration is particularly acute for the Coverage rubric’s post-hoc re-evaluation (§6.3.2, “Per-condition aggregate (Coverage)”), where first-pass scores of 4–5 for Cond A runs were reinterpreted downward to 3 after inspection of the actual generated code. The reinterpretation was applied in good faith and is documented transparently, but the change itself illustrates exactly the kind of judgment-call that blinding would have prevented. Mitigation for future work: each run’s files should be stripped of condition-identifying paths (e.g., `features/flags/` vs. `root lib/`) and re-labelled before scoring, with condition membership recovered only after the scores are committed to `score.json`. The current pilot’s per-run JSON artefacts are preserved so the re-scoring can be done ex post if a second rater becomes available.

7.3 Diffusion Pathway and Adoption Risks

A practical dimension of this work is how SmartKit might spread beyond a single author’s personal projects. Drawing on Rogers’ diffusion of innovations framework (Rogers, 2003) as applied to developer tooling (Storey et al., 2016), the adoption profile of SmartKit depends on five attributes: relative advantage, compatibility, complexity, trialability, and observability.

Relative advantage for the early-majority segment is plausible but unquantified. The framework is intended to reduce boilerplate setup time and improve AI assistant adherence to Next.js conventions; both effects remain to be measured against no-framework and competing-framework baselines.

Compatibility with Next.js is high by construction (SmartKit is a Next.js boilerplate). **Complexity**, however, is substantial: developers adopting SmartKit must internalize feature-based folder conventions, Next.js Server/Client boundary directives (`'use~client'` and `server-only` imports, which are Next.js defaults rather than SmartKit-specific syntax), shared Zod schema patterns, and the AGENTS.md instruction format. This overhead targets the innovator and early-adopter segments of the developer community; whether the framework can spread to the early majority depends on whether the maintenance overhead is amortized across projects.

Trialability is well-addressed: the SmartKit codebase is publicly released, and incremental adoption (taking only the feature-based layout and Server/Client boundary conventions without committing to the full stack) is documented. **Observability** is partially addressed: the proposed three-dimension rubric provides numeric scores per dimension, which can serve as an observability signal in published comparisons.

Adoption risks. Prior work on framework adoption identifies framework lock-in as a major adoption risk (Storey et al., 2016); SmartKit may intensify this risk by introducing a layer of project-specific conventions above plain Next.js. If a future Next.js major release changes its Server/Client boundary conventions (currently `'use~client'` directives and `server-only` imports)—or if the conventions are folded into mainstream documentation—the SmartKit conventions may need to be migrated or retired. Teams that adopt SmartKit incrementally incur a partial lock-in: they may find rewriting their `features/` layout or unwiring shared Zod schemas across feature boundaries to be a non-trivial refactor. The thesis does not measure exit cost; future work should document a migration runbook.

Developer deskilling. A separate concern identified in adjacent work is whether structural boilerplate frameworks erode the developer expertise needed to debug, maintain, or evolve the generated code (Dakhel et al., 2023). Dakhel et al. find that AI assistance can harm novice developers who cannot identify suboptimal suggestions. SmartKit’s explicit, named conventions (“always put Zod schemas in `schemas/`”) are arguably more legible than AI-generated invisible structure, which can partially mitigate this risk: a developer who reads the AGENTS.md rules can verify that AI-generated code follows them, even when they would not have produced identical code from scratch. Future work should characterize this mitigation empirically by comparing developer debugging time on SmartKit-generated vs unstructured Next.js codebases.

7.4 Future Work

Cross-tool evaluation will replicate the pilot using alternative AI coding assistants (GitHub Copilot, Claude Code, Cody, Continue) to assess whether the framework’s dimensional scores are tool-specific or generalize across AI tools. The directional predictions (DP1–DP3) can be tested on each tool, producing a per-tool verdict table. Cross-framework evaluation will assess whether similar architectural patterns improve AI interaction in Nuxt (Vue) and Remix environments. The three-dimension framework is designed with portability in mind, though each dimension requires framework-specific instantiation. **Structural Integrity** maps directly: in Nuxt, the conventions are file-based routing via `/pages/`, `definePageMeta()` for metadata, `useAsyncData()` for server data boundaries, and `.client.vue` suffix for client-only components; in Remix, they are flat routes via `/routes/`, `loader/action` functions as server-side entry points, and `ErrorBoundary` components for error handling. These conventions are structurally analogous to Next.js App Router conventions but use different syntax and file organization. **Type & Contract Safety** transfers directly because it oper-

ates at the language level (TypeScript strict mode) and API level (Zod schemas), which are framework-agnostic. **Coverage & Completeness** requires per-framework checklist instantiation: Nuxt checklists include `middleware/` files and `composables/` for shared logic, while Remix checklists include `utils/` and `styles/` conventions. Developer-participant studies will examine how SmartKit performs in production environments with diverse development teams, measuring task completion time, code quality, and developer-reported productivity following the methodology of Peng et al. (Peng et al., 2023) and Tariq et al. (Tariq et al., 2026). Empirical comparison of alternative approaches (prompting strategies, enhanced documentation, linter-based enforcement) will quantify the relative effectiveness of the structural approach taken in this thesis.

Concrete next steps: (1) run the cross-tool replication on at least two non-Cursor assistants, now that all three DPs have been issued a verdict on the current tool family; (2) extend Cond B from $n = 3$ to $n = 5$ so a symmetric two-condition comparison with a comparative confidence interval becomes possible; (3) instantiate the Nuxt and Remix checklists in draft form for review; (4) pilot a developer-participant session with three developers on a single SmartKit codebase and report findings before scaling further; (5) re-run DP2 with the Cond B prompt removing the shared Zod directive (i.e. instruct Cond B to use raw `string` only) to isolate the structural convention-encoding contribution from the prompting contribution, addressing the bundled-effect limitation disclosed in Section 6.2.

Sample-size determination. The executed pilot ($n = 5$ Cond A after the pre-registered extension; $n = 3$ Cond B) is feasibility-stage, not validation-stage. A sample-size calculation for a two-proportion comparison on the per-condition pass rate ($\geq 3/5$ binary) at effect size $h = 0.5$ (Cohen’s medium effect), $\alpha = 0.05$, and power 0.8 gives $n \approx 30$ per condition (chi-squared approximation). At a stricter effect size $h = 0.3$ (small), the requirement rises to $n \approx 90$ per condition. The binary verdict rule used in the pilot (5/5 confirmed, 0/5 refuted, 1–4/5 inconclusive) is appropriate for a feasibility-stage design at small N , but it is not a substitute for a frequentist sample-size calculation when moving to validation-stage replication: a confirmed verdict at $N = 5$ should not be over-interpreted as a strong inferential claim.

Empirical validation of the orthogonality claim. The three evaluation dimensions (Structural Integrity, Type & Contract Safety, Coverage & Completeness) are claimed in §1.3 to be *designed to be* orthogonal in the sense that they measure distinct aspects of generated code. This orthogonality is a design-stage hypothesis—the three dimensions are intended to vary independently, but this intent has not been empirically tested. A Pearson correlation matrix across the three dimension scores, computed on $n \geq 30$ per condition, would distinguish genuine orthogonality (correlations ≈ 0) from subtle coupling.

Re-execution of the IRR pass. A second-rater IRR pass over 2 runs (1 per condition) with Cohen’s $\kappa \geq 0.7$ is the pre-registered validation step (§6.2.3). The pass was deferred for lack of a second rater within the thesis timeline (Section 7.2). Future work should re-execute the IRR pass once a second rater is available; the rubric-revision clause remains in force but was not exercised.

Bibliography

- [1] Aleem, S., Ahmed, F., Batool, R., & Khattak, A. M. (2019). Empirical investigation of key factors for SaaS architecture. *IEEE Transactions on Cloud Computing*, 9(3), 1037–1049. <https://doi.org/10.1109/TCC.2019.2906299>
- [2] AlOmar, E. A., Venkatakrishnan, A., Mkaouer, M. W., Newman, C. D., & Ouni, A. (2024). How to refactor this code? an exploratory study on developer-ChatGPT refactoring conversations. *Proceedings of the 21st IEEE/ACM International Conference on Mining Software Repositories (MSR)*, 202–206. <https://doi.org/10.1145/3643991.3645081>
- [3] Barke, S., James, M. B., & Polikarpova, N. (2022). Grounded Copilot: How programmers interact with code-generating models. <https://arxiv.org/abs/2206.15000>
- [4] Beller, M., Park, A., Nakad, K., Patel, A., Mohanty, S., Garberson, F., Malone, I. G., Garg, V., Verroken, H., Kennedy, A., & Avgustinov, P. (2026). What’s DAT? measuring software development productivity at Meta. *Proceedings of the 48th International Conference on Software Engineering (ICSE)*. <https://arxiv.org/abs/2503.10977>
- [5] Chen, M., Tworek, J., Jun, H., et al. (2021). Evaluating large language models trained on code. <https://arxiv.org/abs/2107.03374>
- [6] Dakhel, A. M., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., & Jiang, Z. M. J. (2023). Github Copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*, 203, 111734. <https://doi.org/10.1016/j.jss.2023.111734>
- [7] Hendrycks, D., Basart, S., Kadavath, S., et al. (2021). Measuring coding challenge competence with APPS. *Proceedings of the 35th Conference on Neural Information Processing Systems (NeurIPS)*. <https://arxiv.org/abs/2105.09938>
- [8] Hou, X., Zhao, Y., Liu, Y., Yang, Z., Wang, K., Li, L., Luo, X., Lo, D., Grundy, J., & Wang, H. (2024). Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology*, 33(8), 1–79. <https://doi.org/10.1145/3695988>
- [9] Inayatullah, I., Azam, F., & Anwar, M. W. (2019). Model-based scaffolding code generation for cross-platform applications. *Proceedings of the 2019 IEEE 10th Annual Information Technology, Electronics and Mobile Computing Conference (IEMCON)*. <https://doi.org/10.1109/IEMCON.2019.8936289>
- [10] Jain, N., Han, K., Gu, A., et al. (2024). Livecodebench: Holistic and contamination free evaluation of large language models for code. *Proceedings of the 13th International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2403.07974>
- [11] Jimenez, C., Yang, J., Gletner, A., et al. (2024). Swebench: Can language models resolve real-world github issues? <https://arxiv.org/abs/2212.09741>

- [12] Krüger, J., & Berger, T. (2020). An empirical analysis of the cost of clone- and platform-oriented software reuse. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 1103–1113. <https://doi.org/10.1145/3368089.3409684>
- [13] Li, Z., Lu, S., Guo, D., et al. (2022). Automating code review activities by large-scale pre-training. *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. <https://arxiv.org/abs/2203.09095>
- [14] Liu, J., Tang, T., Du, Z., et al. (2024). Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. <https://arxiv.org/abs/2305.01210>
- [15] Liu, Y., Le-Cong, T., Widyasari, R., et al. (2024). Refining ChatGPT-generated code: Characterizing and mitigating code quality issues. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3678167>
- [16] Liu, Y., & Sra, M. (2025). Auggen: Augmenting task-based learning in professional creative software with LLM-generated scaffolded UIs. <https://arxiv.org/abs/2511.23379>
- [17] Liu, Z., Tang, Y., Luo, X., Zhou, Y., & Zhang, L. F. (2024). No need to lift a finger anymore? assessing the quality of code generation by ChatGPT. <https://doi.org/10.1109/TSE.2024.3392499>
- [18] Madiraju, M. B., & Madiraju, M. S. P. (2026). Rigorbench: Benchmarking engineering process discipline in autonomous AI coding agents. <https://arxiv.org/abs/2606.22678>
- [19] Nguyen-Duc, A., et al. (2025). Generative artificial intelligence for software engineering—a research agenda. *Software: Practice and Experience*. <https://doi.org/10.1002/spe.70005>
- [20] Ouyang, S., et al. (2025). An empirical study of the non-determinism of ChatGPT in code generation. *ACM Transactions on Software Engineering and Methodology*. <https://doi.org/10.1145/3695989>
- [21] Pandey, R., et al. (2024). Transforming software development: Evaluating Copilot in real-world projects. <https://arxiv.org/abs/2406.17910>
- [22] Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. <https://arxiv.org/abs/2302.06590>
- [23] Petrukha, I., Kurliak, Y., & Stulova, N. (2025). Swifteval: Developing a language-specific benchmark for LLM-generated code evaluation. *Proceedings of IEEE/ACM 3rd International Conference on Software Engineering Education and Training (FORGE)*. <https://arxiv.org/abs/2505.24324>
- [24] Rogers, E. M. (2003). *Diffusion of innovations* (5th). Free Press.

- [25] Russo, D., et al. (2024). Navigating the complexity of generative AI adoption in software engineering. *ACM Transactions on Software Engineering and Methodology*, 33(5), 1–50. <https://doi.org/10.1145/3652154>
- [26] Sauvola, T., et al. (2024). Future of software development with generative AI. *Journal of Automated Software Engineering*. <https://doi.org/10.1007/s10515-024-00426-z>
- [27] Stack Overflow. (2025). Developer survey 2025 [Accessed: 2026-09-08]. <https://survey.stackoverflow.co/2025/>
- [28] Storey, M.-A., Zimmermann, B., Beschonick, F., et al. (2016). The adoption of software development framework: A family of patterns. *Journal of Systems and Software*, 122, 110–127. <https://doi.org/10.1016/j.jss.2016.09.022>
- [29] Tariq, M., Hassan, A., & Yazdani, M. F. (2026). An empirical evaluation of AI-assisted code generation on developer productivity and code quality: A case study of .net applications. <https://doi.org/10.2139/ssrn.6932038>
- [30] Vercel. (2024). Next.js documentation - app router introduction [Accessed: 2026-09-09; Next.js 16.2.12 confirmed in SmartKit/package.json]. <https://nextjs.org/docs>
- [31] Yetiştiren, B., Özsoy, I., Ayerdem, M., & Tüzün, E. (2023). Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. <https://arxiv.org/abs/2304.10778>
- [32] Yetiştiren, B., Ozsoy, I., & Tüzün, E. (2022). Assessing the quality of GitHub Copilot’s code generation. <https://doi.org/10.1145/3558489.3559072>
- [33] Yu, Z., Zhao, Y., Cohan, A., & Zhang, X.-P. (2025). Humaneval pro and MBPP pro: Evaluating large language models on self-invoking code generation task. *Findings of the Association for Computational Linguistics: ACL 2025*, 13253–13279. <https://doi.org/10.18653/v1/2025.findings-acl.686>
- [34] Zhang, B., Liang, P., Zhou, X., Ahmad, A., & Waseem, M. (2023). Practices and challenges of using GitHub Copilot: An empirical study. <https://arxiv.org/abs/2303.08733>
- [35] Zhuo, T. Y., Chien, V. Y., Chim, J., et al. (2024). Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *Proceedings of the 12th International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/2406.15877>